Machine Learning

Lecture 10: Transformer

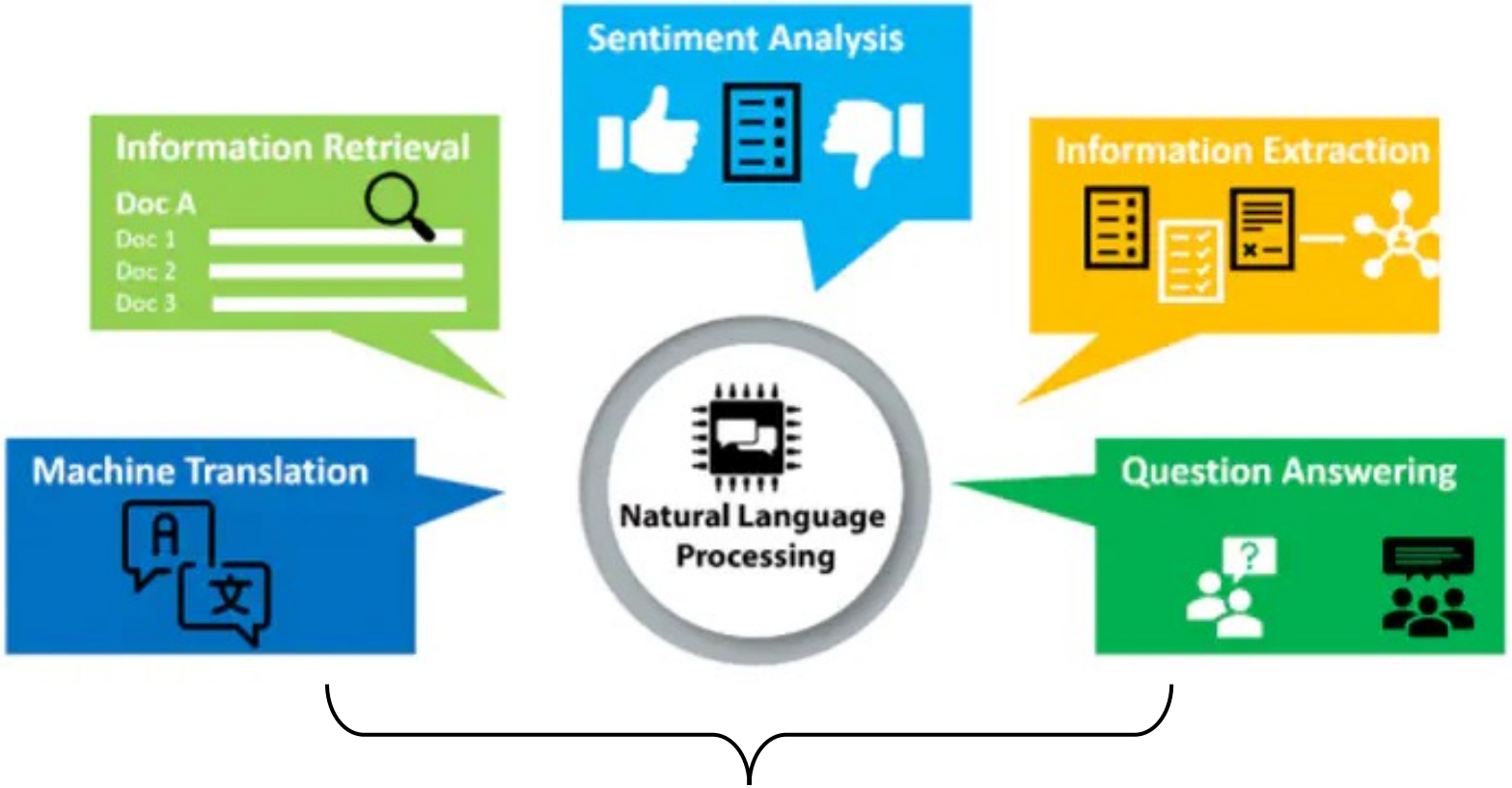
Spring 2025

Instructor: Xiaodong Gu



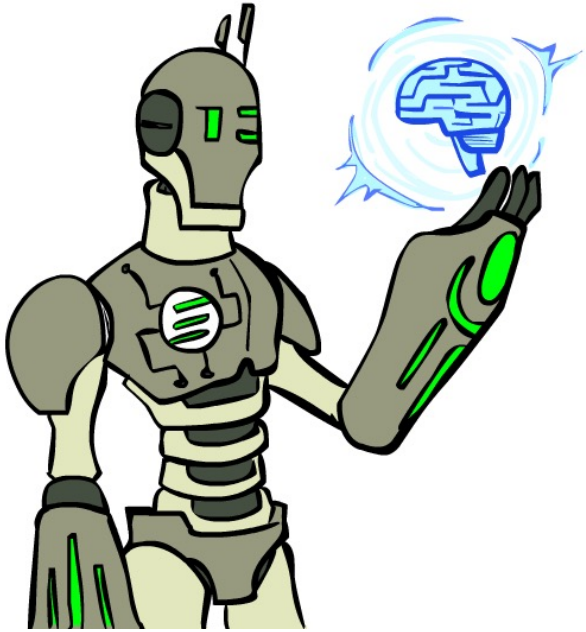
Natural Language Processing (NLP)

Language Understanding and Generation



Today

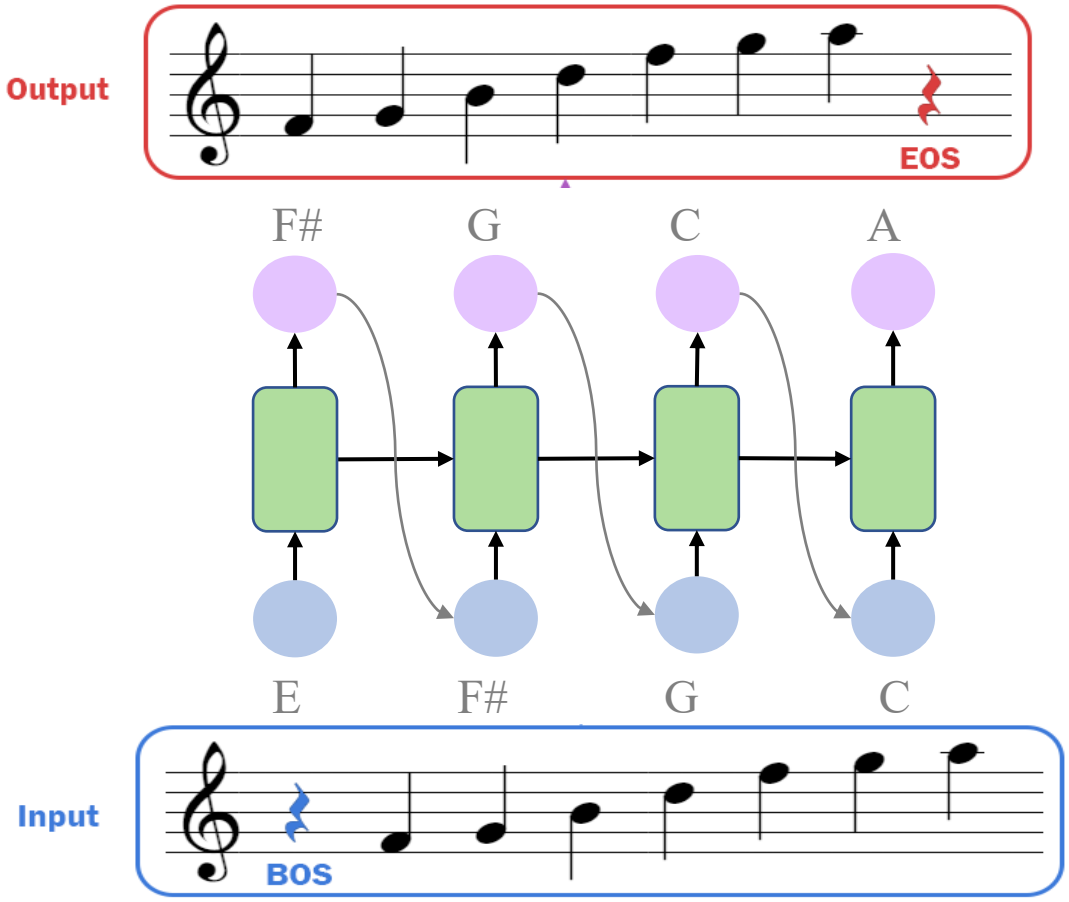
- RNN Encoder-Decoder
- Attention
- Transformer





Sequence-to-Sequence Learning

Recall: Sequence Generation Using RNNs



Conditional Sequence Generation

- generate a (**target**) sequence given a (**source**) sequence.

ASR



Generator

How are you

Translation

机器学习

Generator

Machine Learning

Dialogue

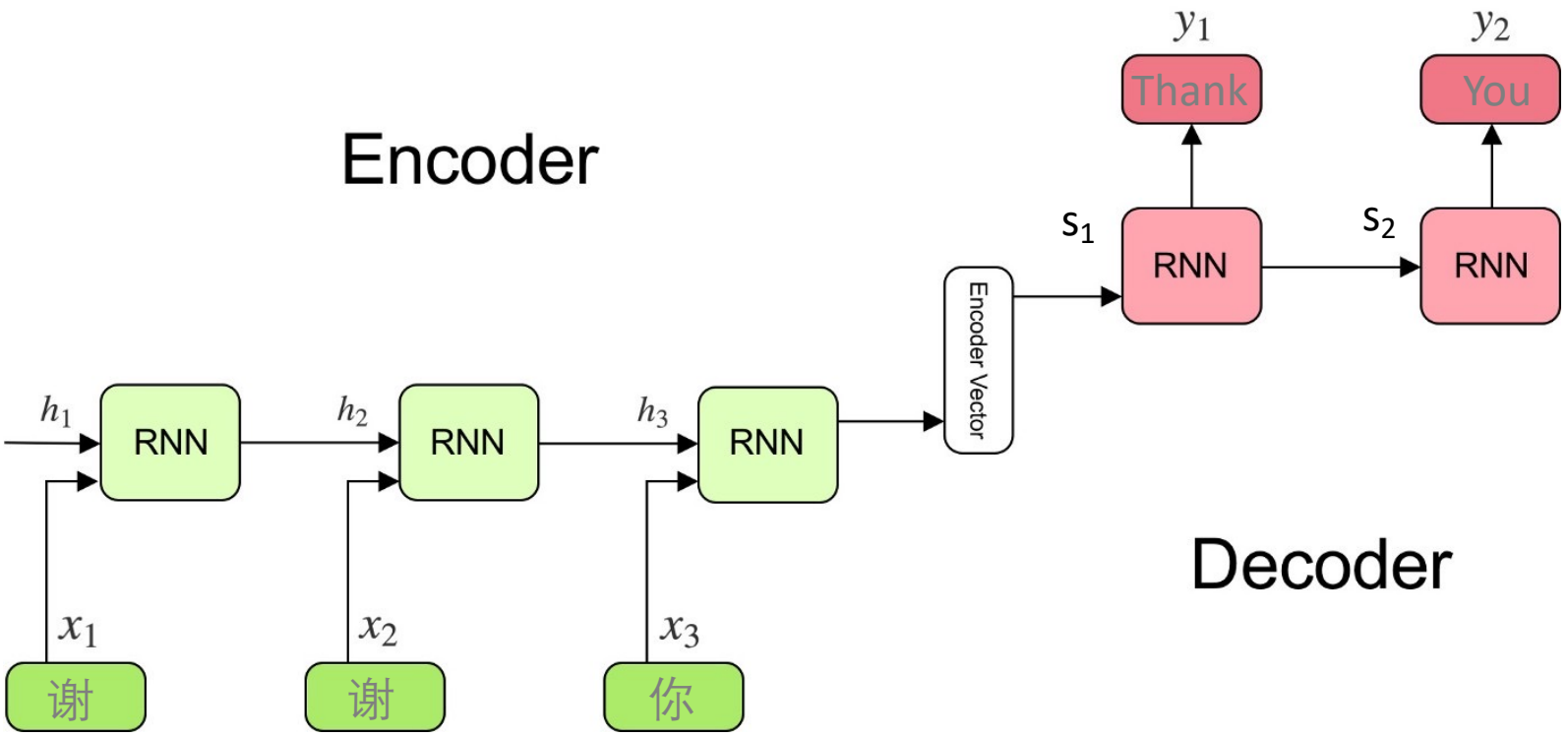
How are you

Generator

I am fine

RNN Encoder-Decoder (Cho 2014)

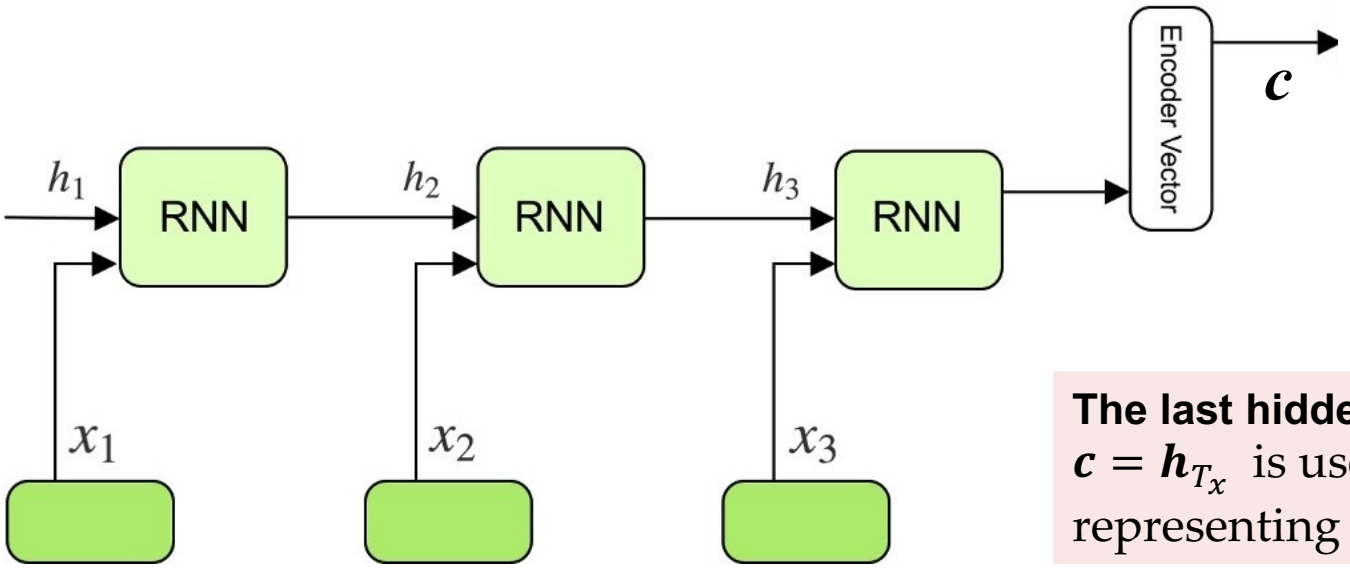
- given $x = (x_1, \cdots, x_{T_x})$, generate $y = (y_1, \cdots, y_{T_y})$



Encoder

- An RNN which learns a dense representation of a sequence.
- **Compresses** a sequence of tokens into a context vector c.

$$\boldsymbol{h}_t = \mathrm{RNN}_{\mathrm{in}}(x_t, \boldsymbol{h}_{t-1})$$



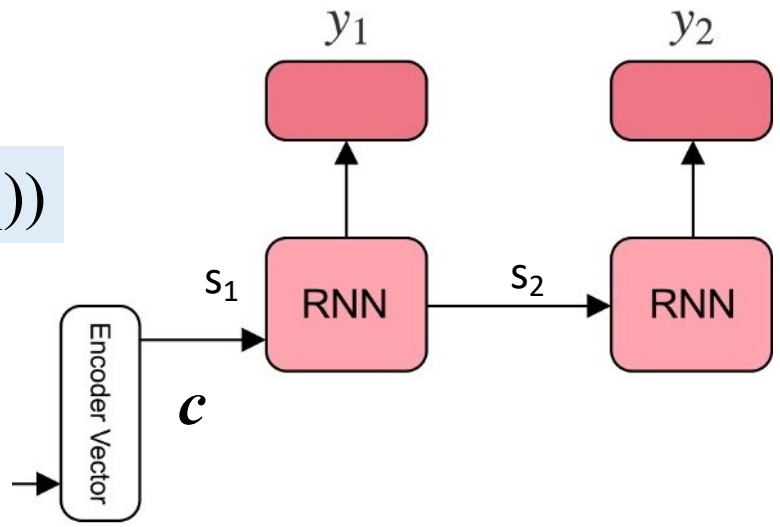
The last hidden state $\boldsymbol{c} = \boldsymbol{h}_{T_x}$ is used for representing x

Decoder

- An RNN which generates an sequence **conditioned on** the intermediate representation.
- Sequentially predicts the next token y_i given the context vector $\boldsymbol{c}$ and the hidden state of past-generated sequence.

$$\boldsymbol{s}_i = \text{RNN}_{\text{out}}(y_{i-1}, \boldsymbol{s}_{i-1}, \boldsymbol{c})$$

$$p(y_i \mid y_{<i}, \boldsymbol{c}) = \text{softmax}(\text{MLP}(\boldsymbol{s}_i))$$



Training

Data: $D = \{(x^{(1)}, y^{(1)}), \ldots, (x^{(N)}, y^{(N)})\}$,

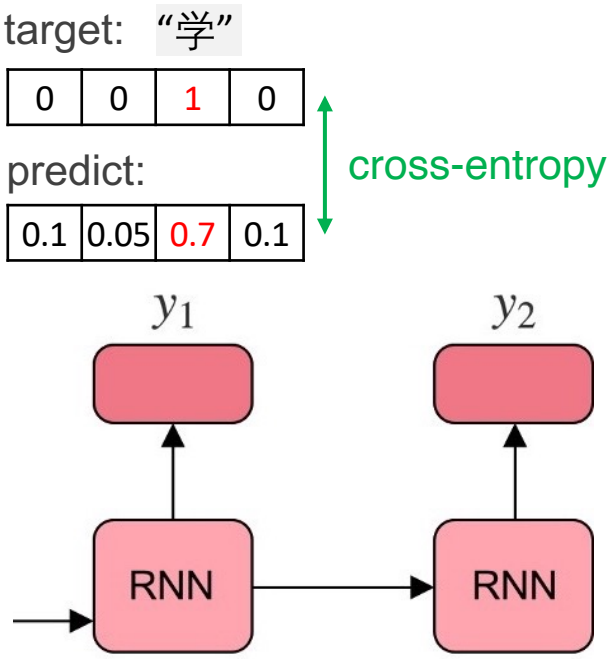
where $x^{(\ell)} = (x_1, \ldots x_{Tx})$ and $y^{(\ell)} = (y_1, \ldots y_{Ty})$.

Loss Function – minimize the **cross-entropy** loss:

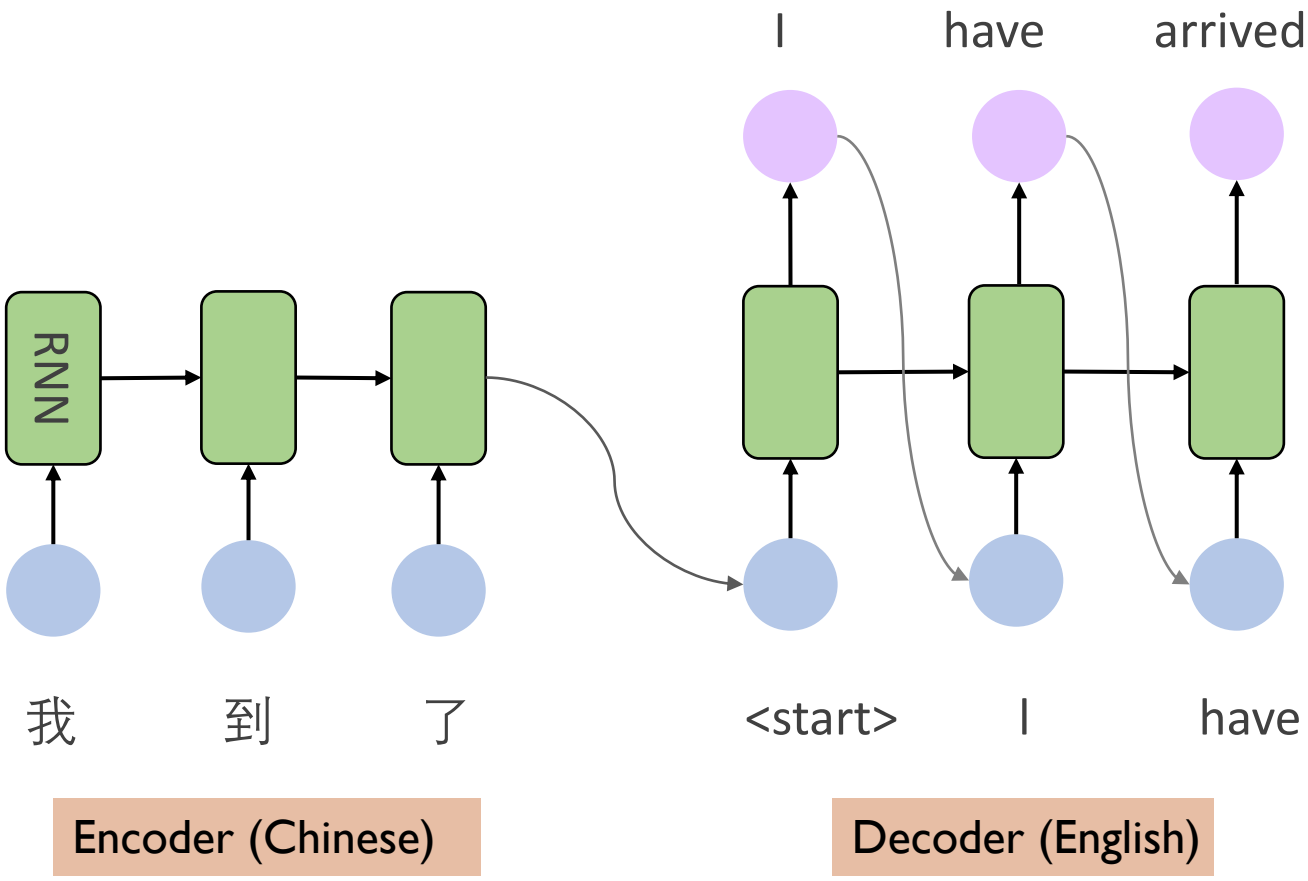
$$l(\theta|x, y) = -\sum_{t=1}^{T} \log p_\theta(y_t|y_1, \ldots, y_{t-1}, x)$$

$$L(\theta|D) = -\frac{1}{N}\sum_{l=1}^{N} l(\theta|x^{(l)}, y^{(l)})$$

Optimization – gradient descend



Example: Machine Translation



TIME for Some Coding



Tutorial: machine translation

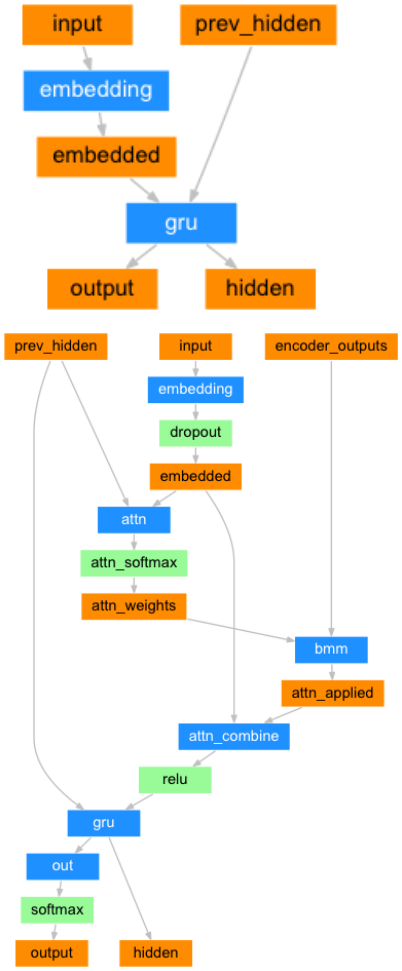
https://pytorch.org/tutorials/intermediate/seq2seq_translation_tutorial.html



Tutorial: Machine Translation

```
class Encoder(nn.Module):
    def __init__(self, input_size, hidden_size):
        self.embedding = nn.Embedding(input_size, hidden_size)
        self.rnn = nn.LSTM(hidden_size, hidden_size)
    def forward(self, input, hidden):
        embedded = self.embedding(input).view(1, 1, -1)
        output, hidden = self.rnn(embedded, hidden)
        return output, hidden
```

```
class Decoder(nn.Module):
    def __init__(self, hidden_size, vocab_size):
        self.embedding = nn.Embedding(output_size, hidden_size)
        self.rnn = nn.LSTM(hidden_size, hidden_size)
        self.out = nn.Linear(hidden_size, vocab_size)
    def forward(self, input, c, hidden):
        input = self.embedding(input).view(1, 1, -1)
        output, hidden = self.rnn([input,c], hidden)
        output = self.out(output[0]))
        return output, hidden
```



Applications

Translation

Machine Learning

RNN Decoder

RNN Encoder

机 器 学 习

Conversation

I am good

RNN Decoder

RNN Encoder

How are you

.....

Image Captioning

A cat standing on the grass

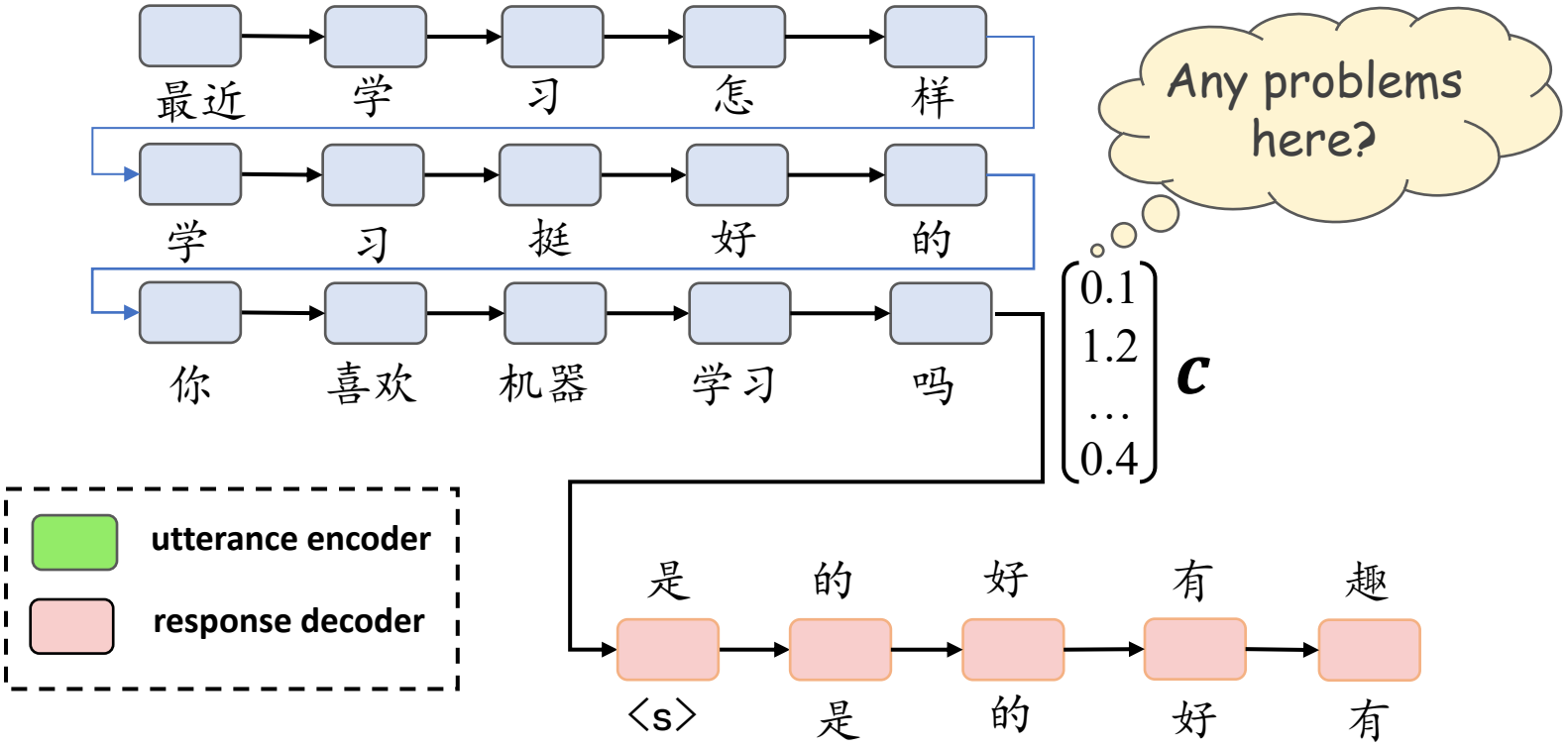
RNN Decoder

CNN Encoder



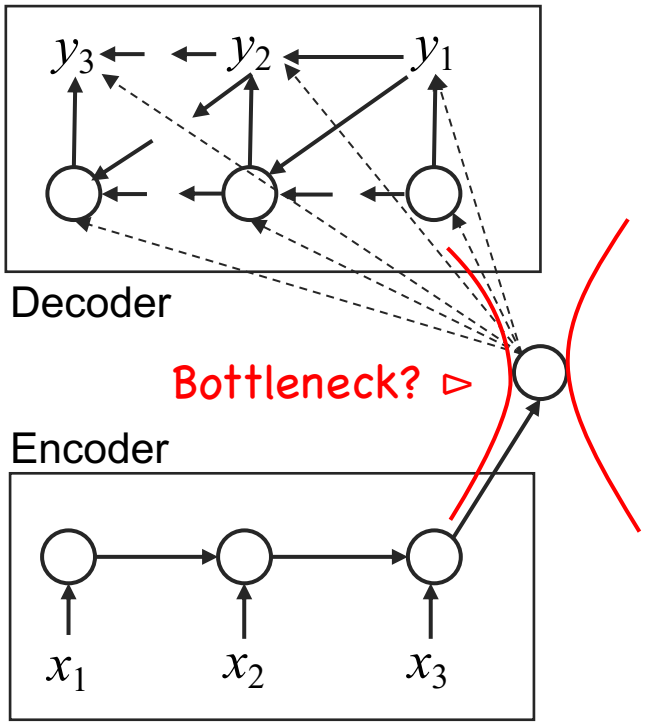
When Language Model Meets Long Sequence

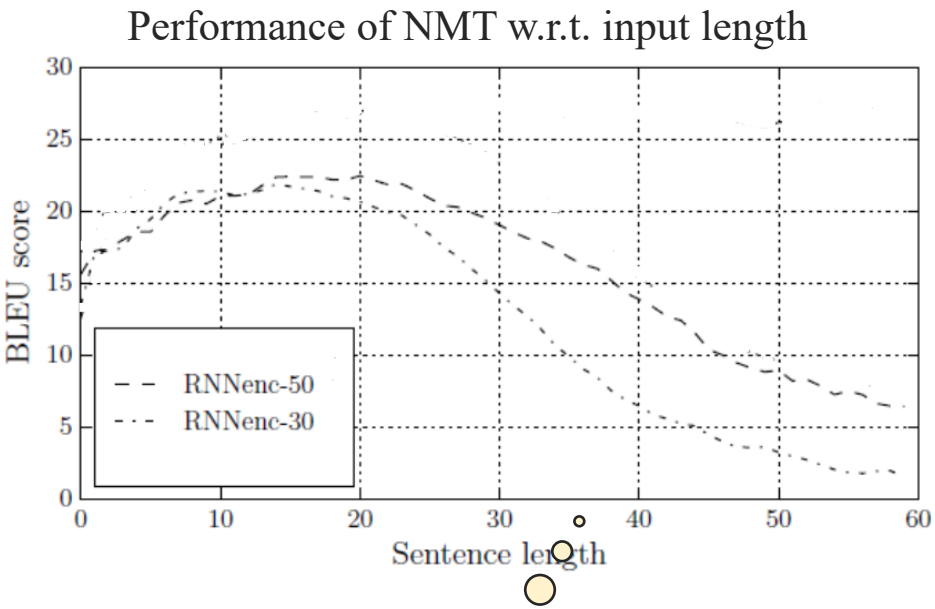
Example: RNN Encoder-Decoder for Long Dialogues



Intermediate Representation: One Size Fits All?

Limitation: the entire sequence is compressed into **one** vector, regardless of the importance of each position.





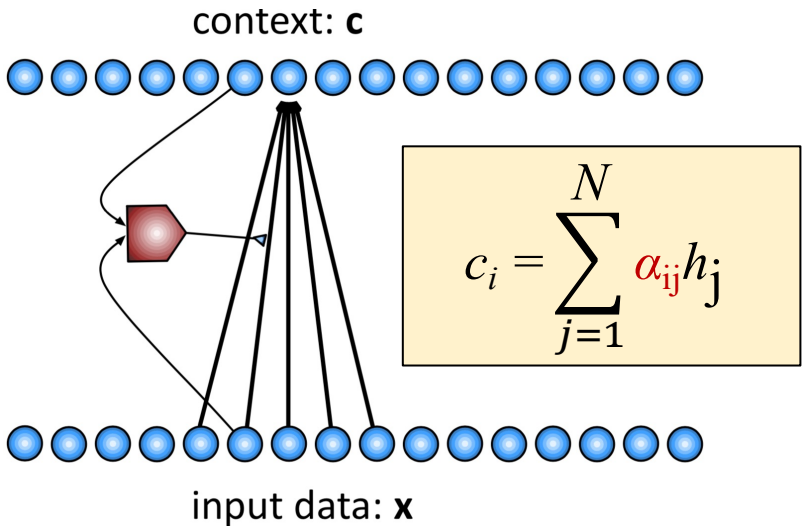
Performance decreases as the sequence length grows.

How to Mitigate "the Curse of Length"?

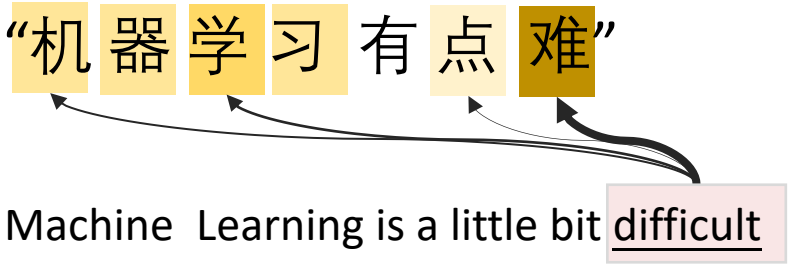
Idea: use dynamic context vectors



- While decoding each y_i in $y = (y_1, \cdots, y_T)$, use a **dynamic context vector** c_i which is computed as a linear combination of hidden states at different positions of x.



$$c_i = \sum_{j=1}^{N} \alpha_{ij} h_j$$

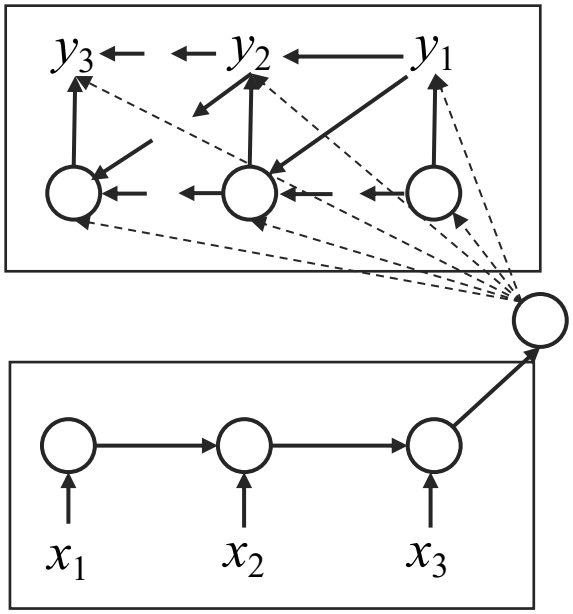


Sequence-to-Sequence with Attention

No Attention

decoding based on a single, fixed encoding vector

$h_i = \text{RNN}_{\text{out}}(h_{i-1}, y_{i-1}, \boldsymbol{c})$

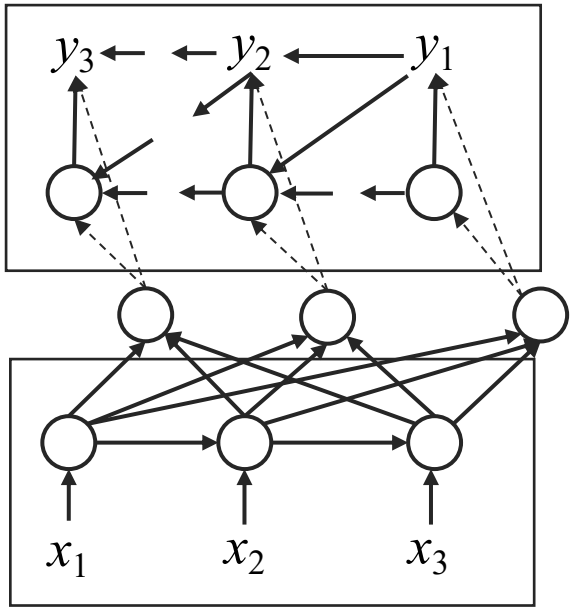


VS

With Attention

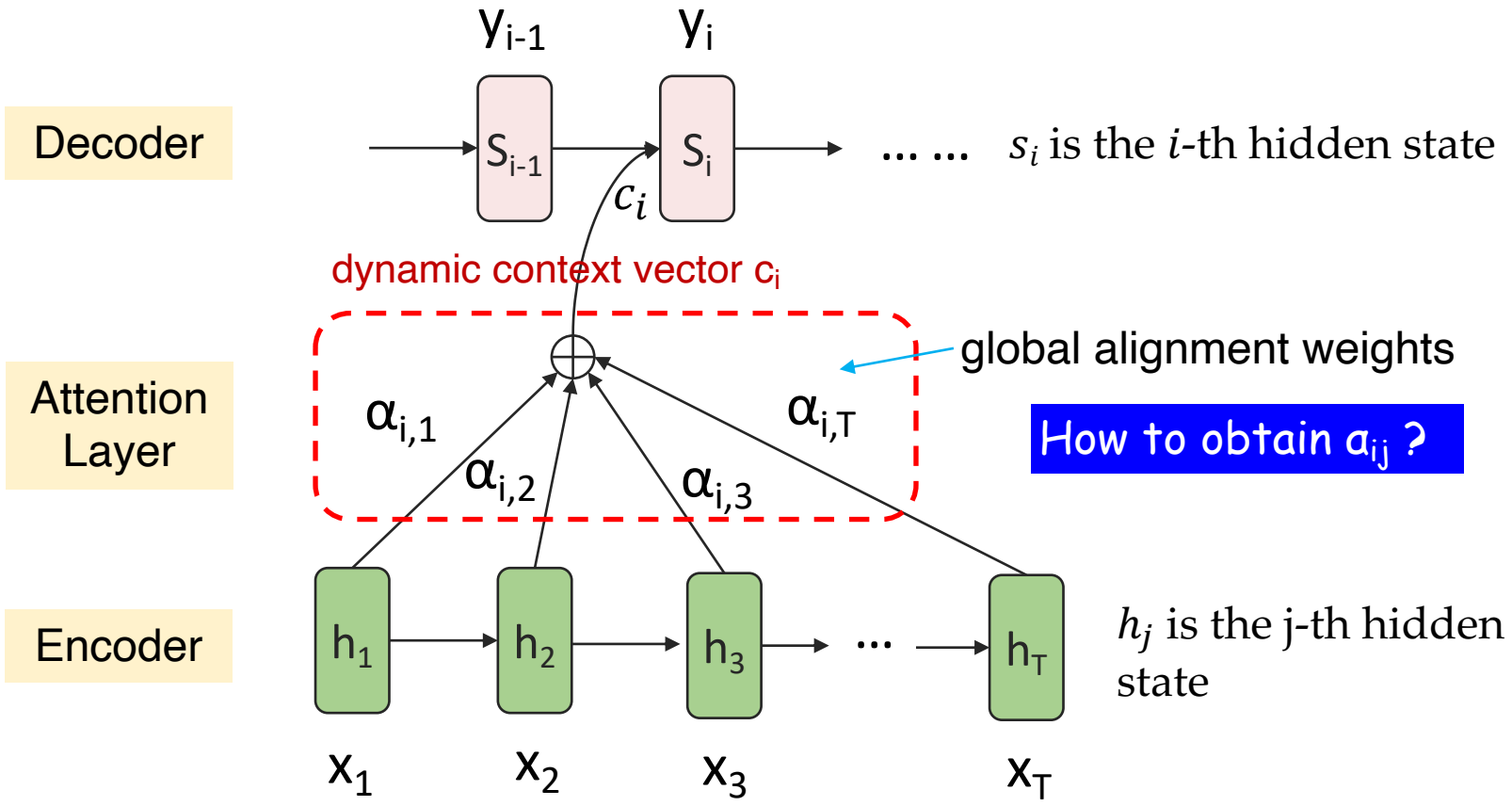
select what to encode in a contextual manner.

$h_i = \text{RNN}_{\text{out}}(h_{i-1}, y_{i-1}, \boldsymbol{c_i})$



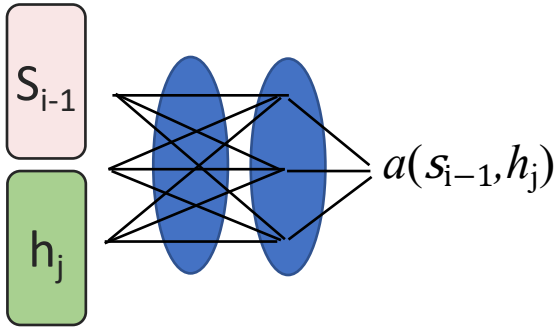
Sequence-to-Sequence with Attention

The decoder **dynamically** pays attention to different tokens in the source sentences during decoding.



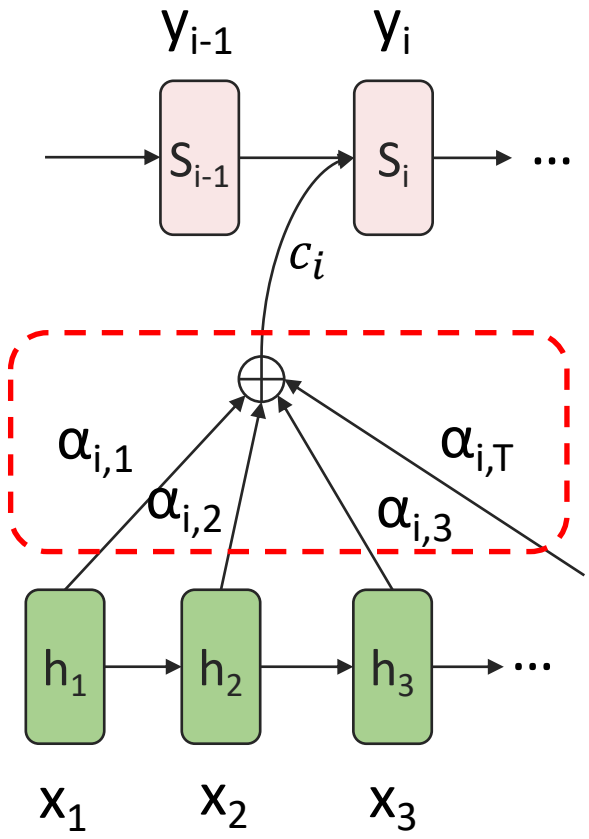
Attention-based Model

- Let $a_{i,j}$ scores how well the input state $\mathbf{h}_j$ and the output state $\boldsymbol{s}_i$ match.
- can be estimated using a **neural network**: e.g., $a_{i,j}=NN(\boldsymbol{s}_{i-1}, \boldsymbol{h}_j)$

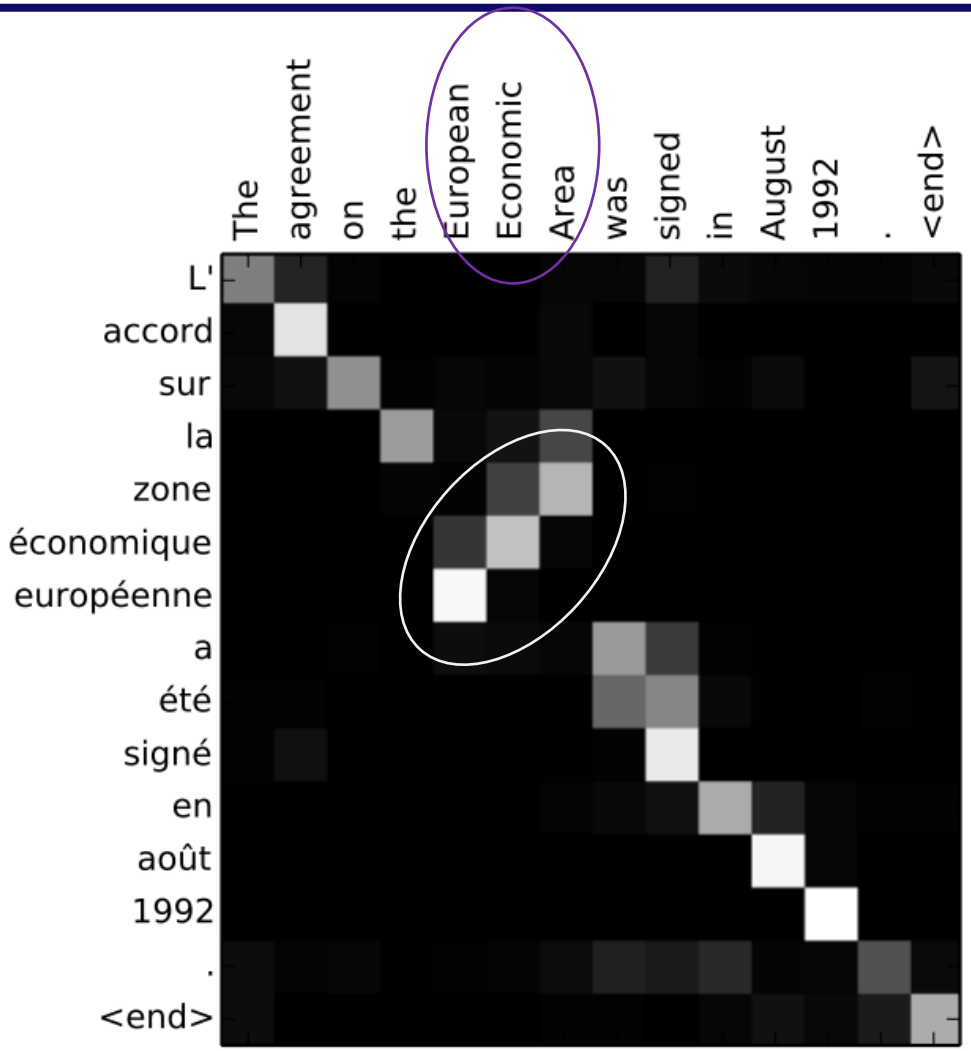


- Further normalize them into probabilities using softmax:

$$\alpha_i = \text{softmax}(a_{i,1}, a_{i,2},\ldots,a_{i,T})$$



Visualizing Attention



real α_{ij}'s in machine translation

You can see how the model paid attention correctly when outputting "European Economic Area". In French, the **order of these words is reversed** ("européenne économique zone") as compared to English. Every other word in the sentence is in similar order.

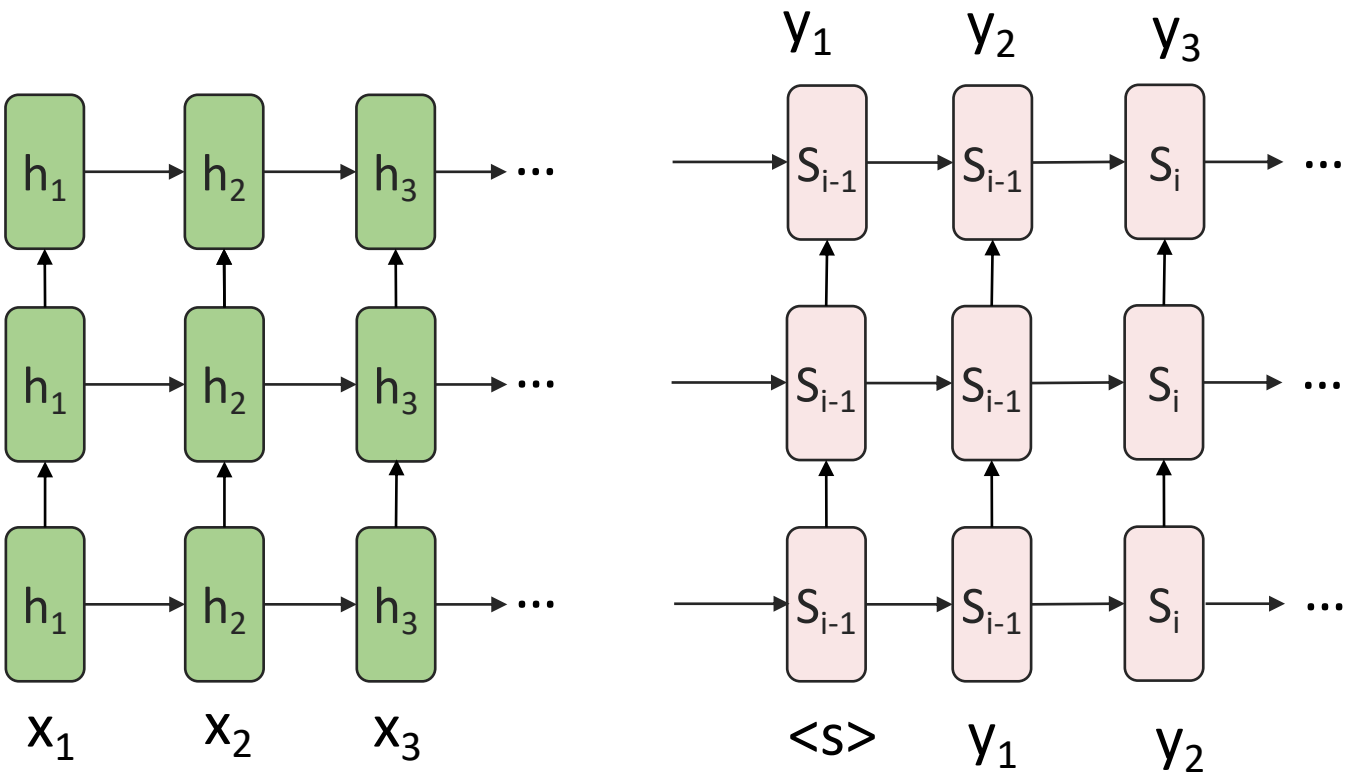


Transformer

Seq2seq model with "Self-attention"

Limitations of RNN

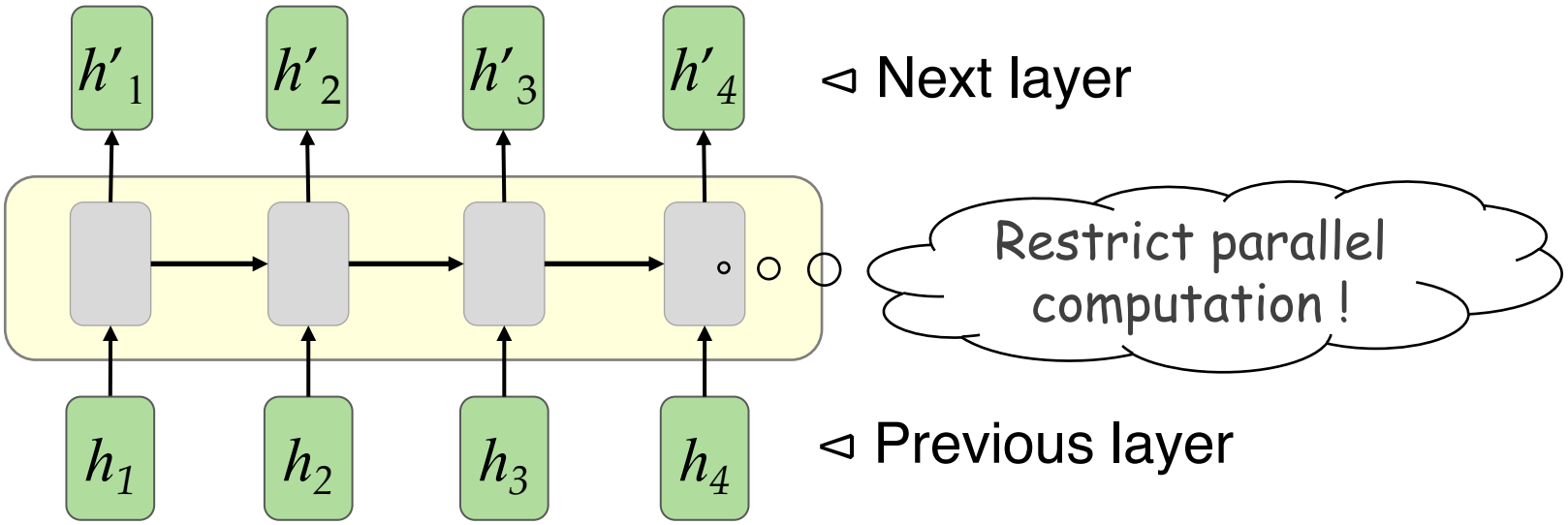
- Multilayer RNNs



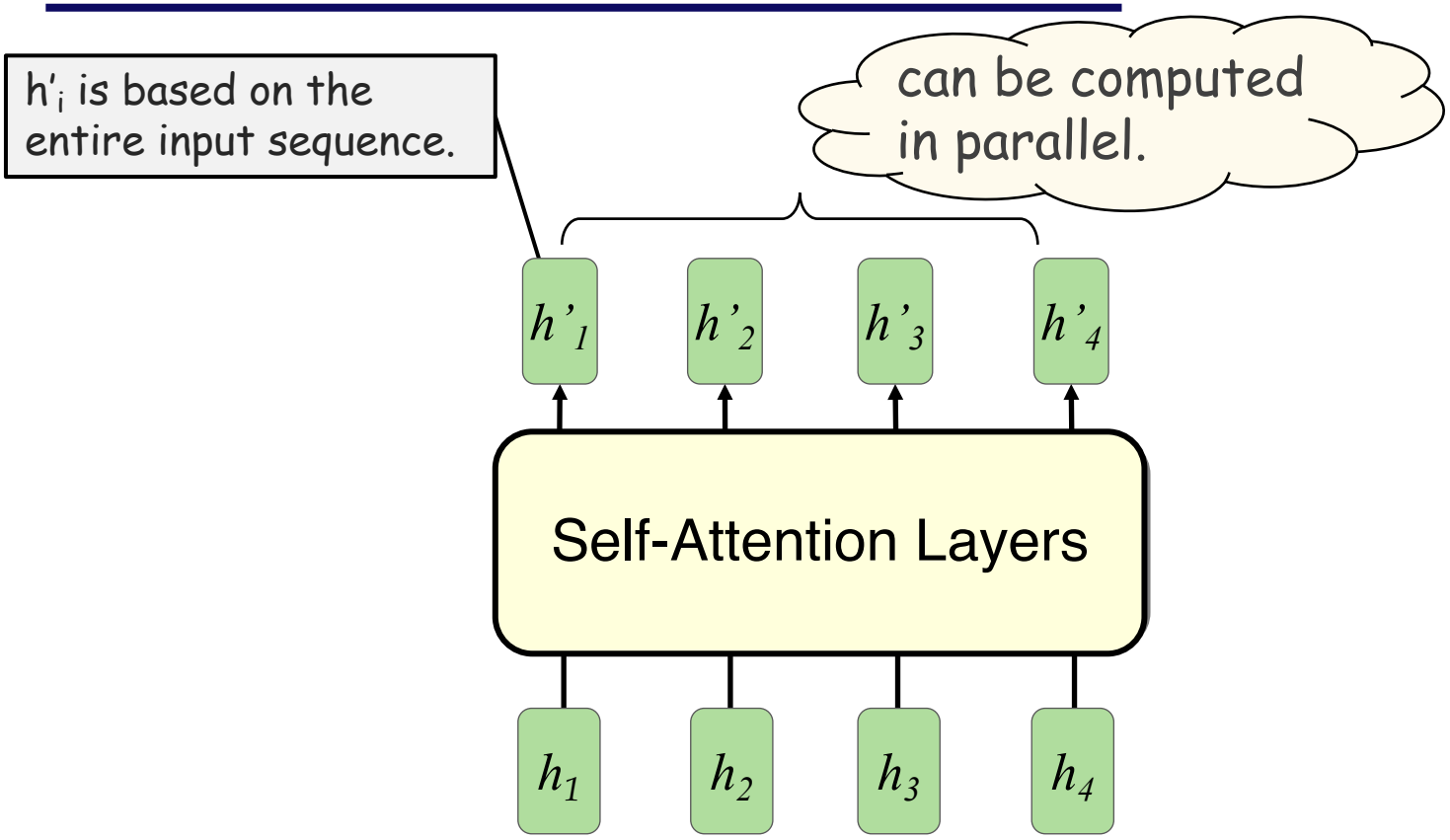
Q: How many *for* loops ?

Limitations of RNN

- **Problem :** the hidden state for each position relies on the computation of the preceding position.



From RNN to Self-Attention





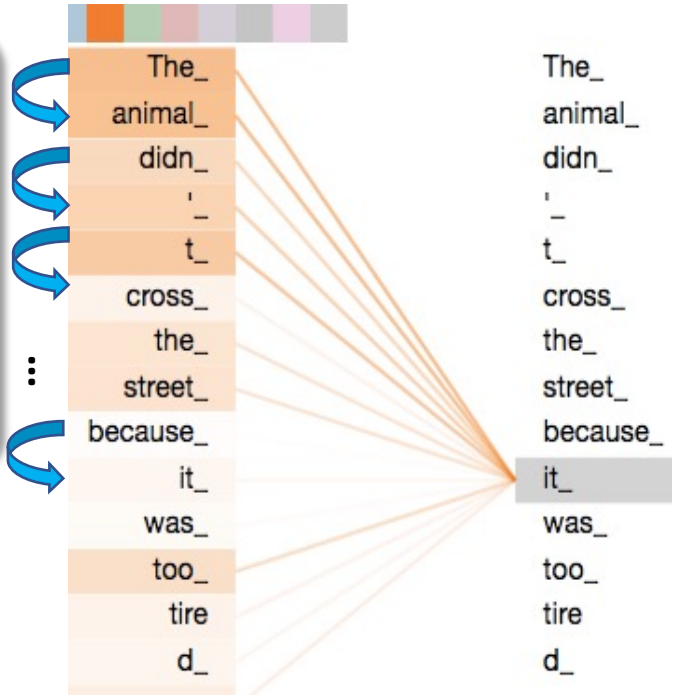
Farewell RNN: We can replace anything that can be processed by an RNN with a self-attention network.

From RNN to Self-Attention

What does "**it**" in this sentence refer to?

RNN

maintain a hidden state to sequentially incorporate previous words with the current one it's processing.

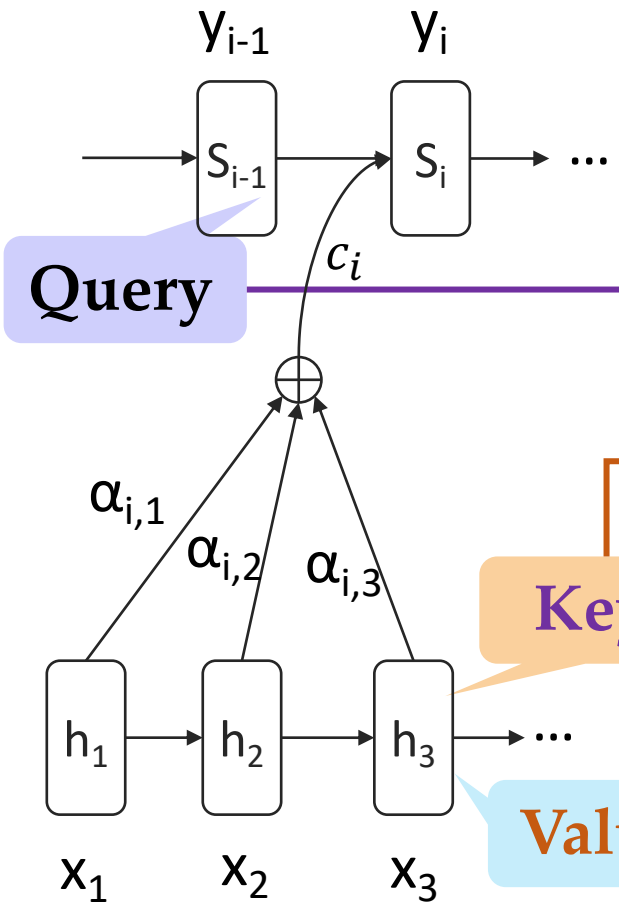


Self-Attention

bake the "understanding" of other relevant words into the one we're currently processing simultaneously.

The animal didn't cross the street because **it** was too tiered.

Attention Revisited



q: query (to match others)

k: key (to be matched)

v: content to be retrieved

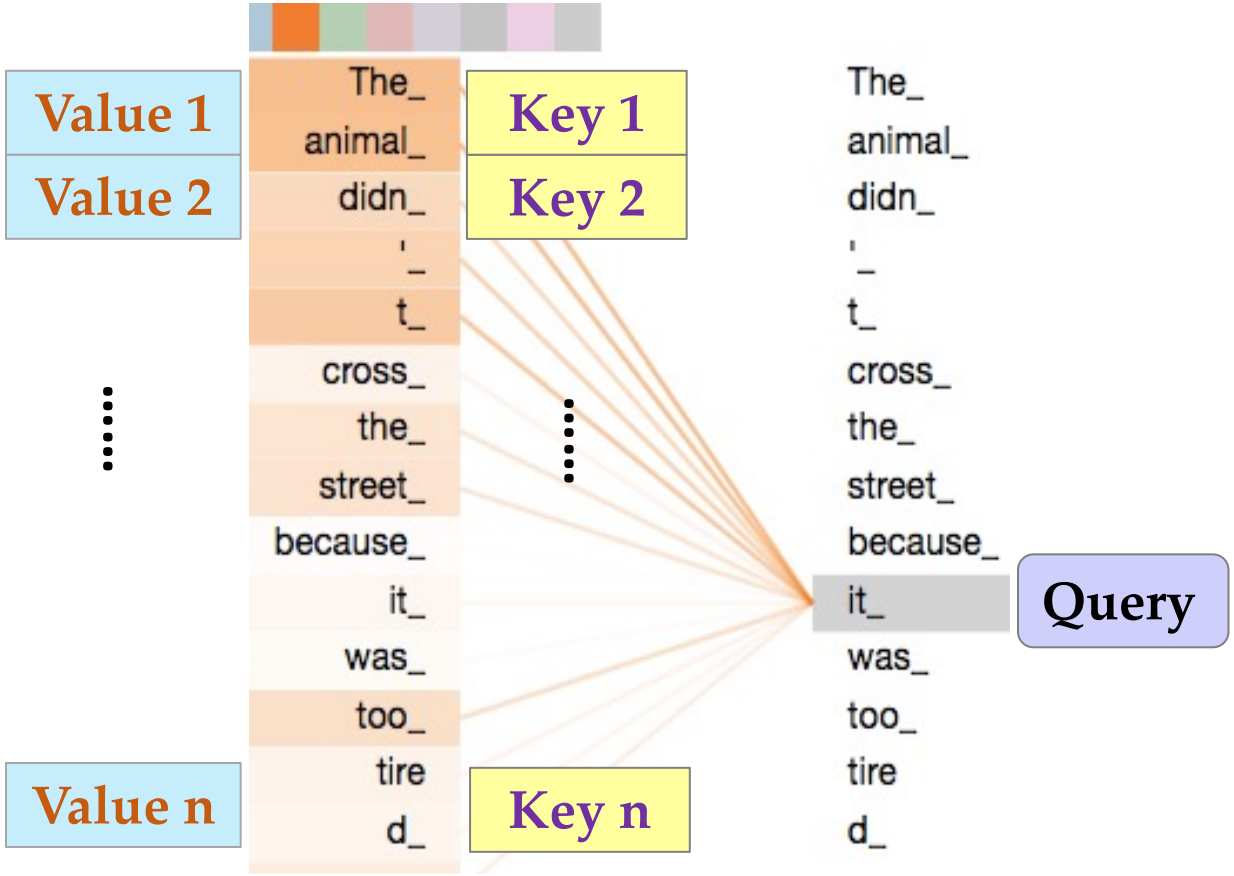
$$a_{ij} = NN(s_{i-1}, h_j)$$

$$\alpha_i = \text{softmax}(a_{i,1},\ldots,a_{i,T})$$

$$c_i = \sum_{j=1}^{Tx} \alpha_{ij} h_j$$

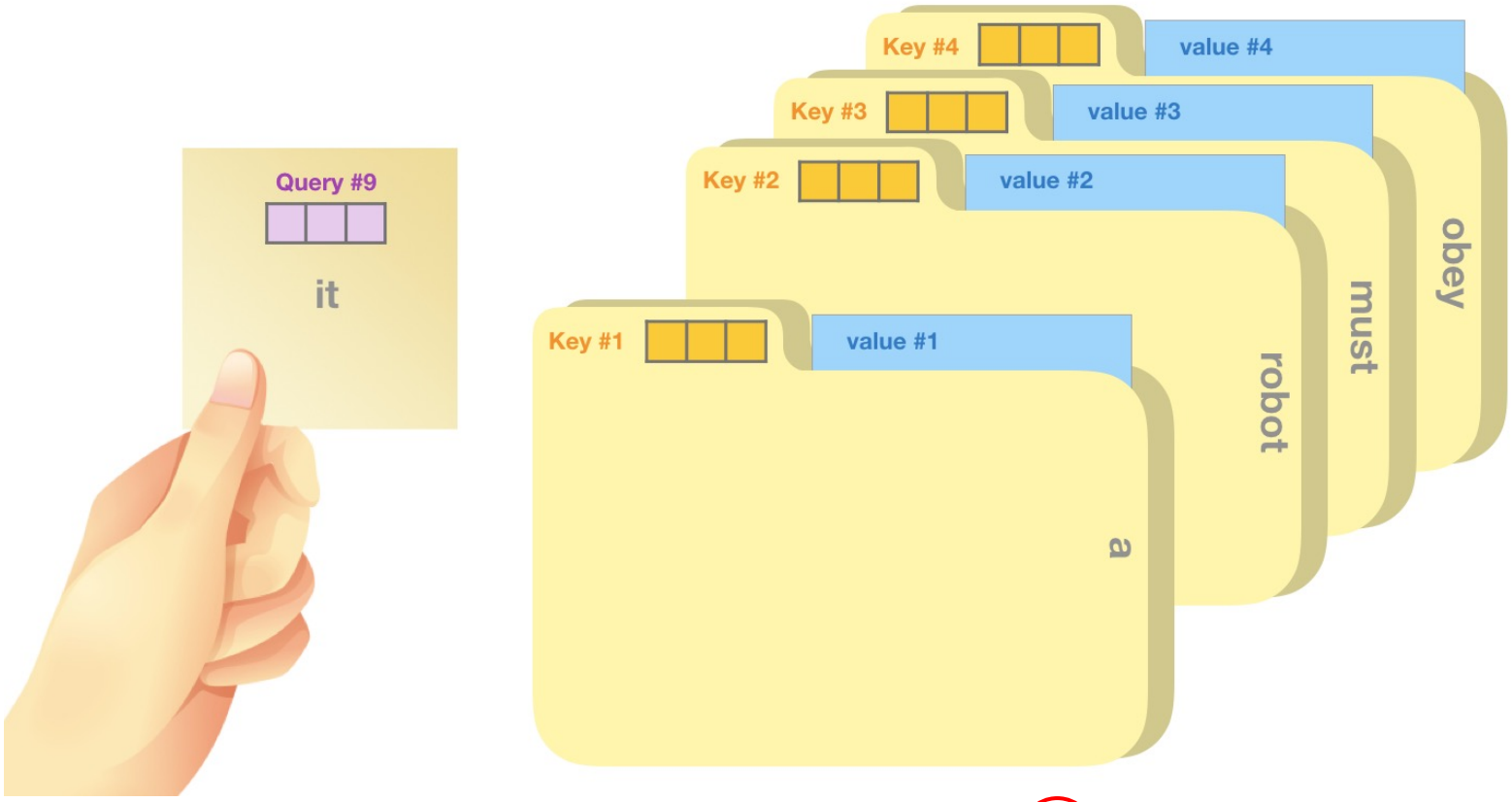
Self-Attention: The Idea

- Each word pays attention to all other words.



Self-Attention: The Idea

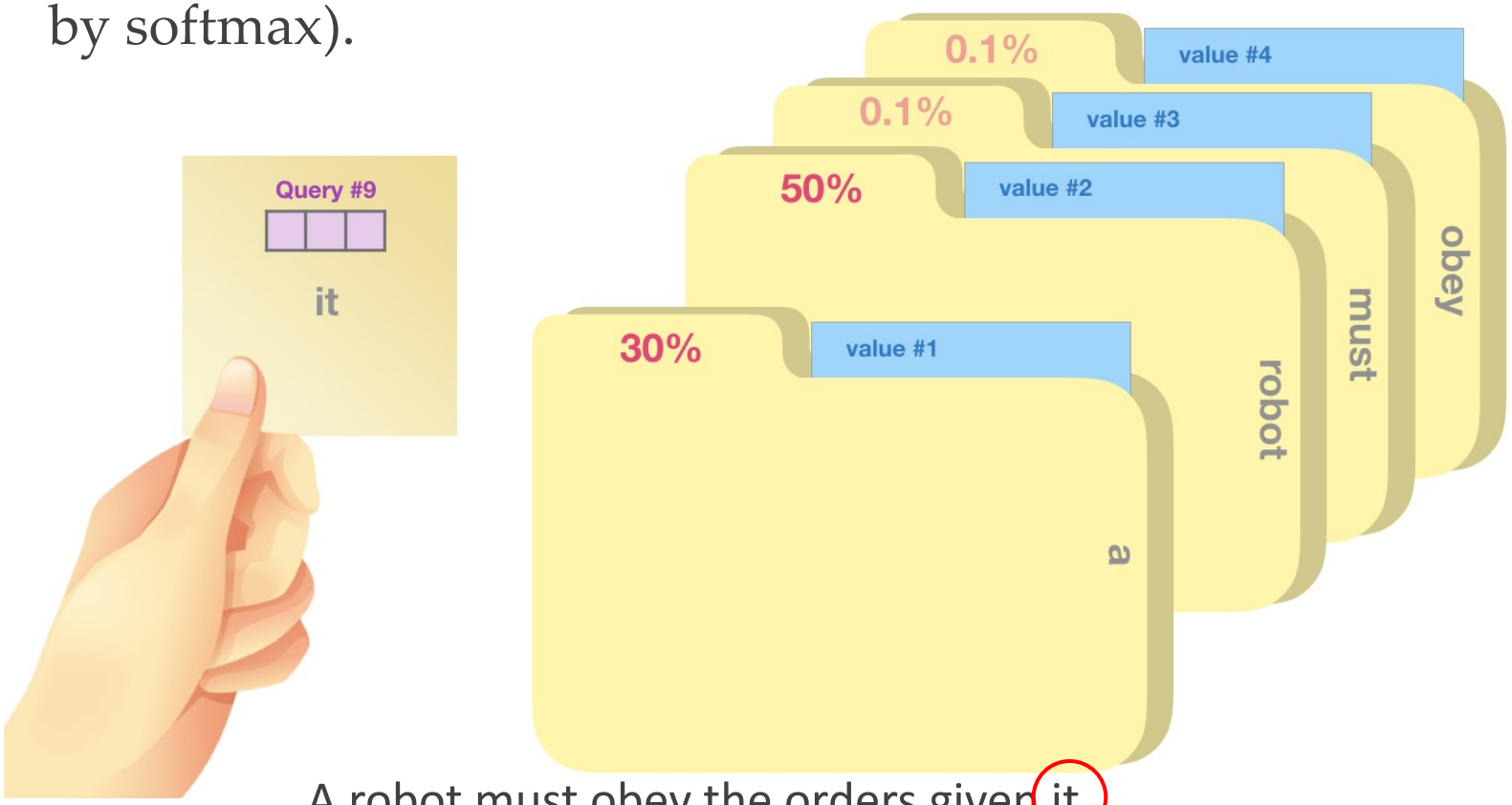
- Let each word pay attention to all other words.



A robot must obey the orders given it.

Self-Attention: the Idea

- Multiplying the query vector by each key vector produces a score for each value (technically: dot product followed by softmax).



A robot must obey the orders given it.

Self-Attention: the Key Idea

- We multiply each value by its score and sum them up.

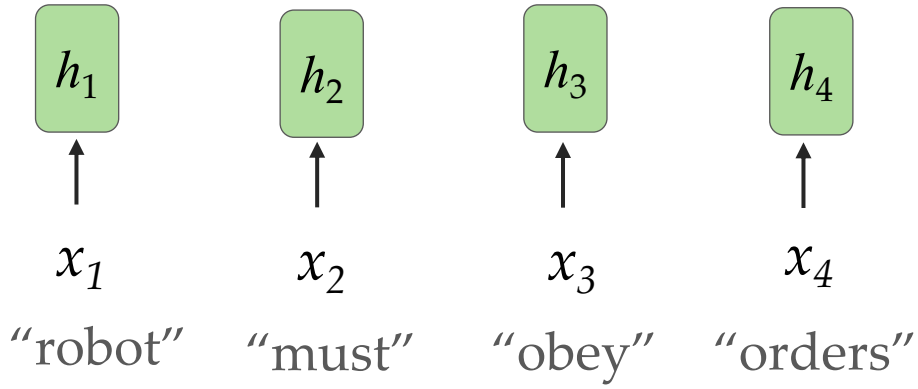
Word	Value vector	Score	Value X Score
<s>		0.001	
a		0.3	
robot		0.5	
must		0.002	
obey		0.001	
the		0.0003	
orders		0.005	
given		0.002	
it		0.19	
		Sum:	

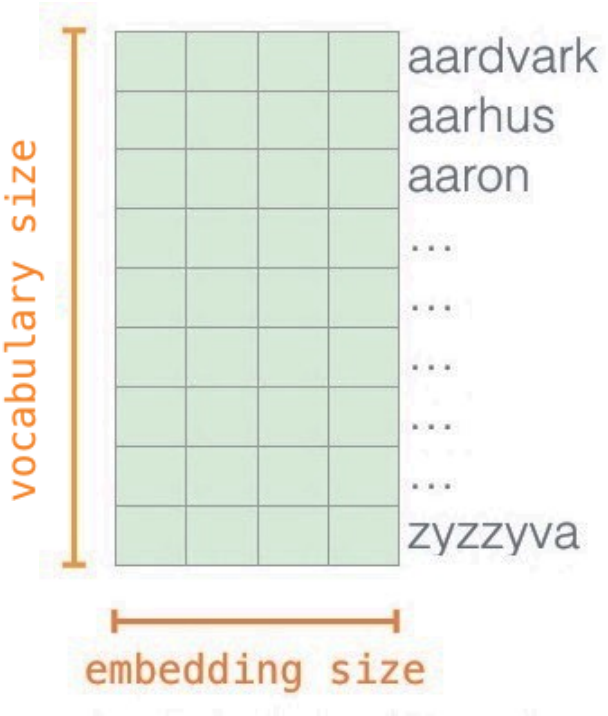
- The outcome vector represents the new state (refreshed memory) for the query word.

Step 1: Token Embedding

- Embedding tokens (integer id) into vectors (hidden states)

$$\boldsymbol{h}_i = \boldsymbol{W} x_i$$

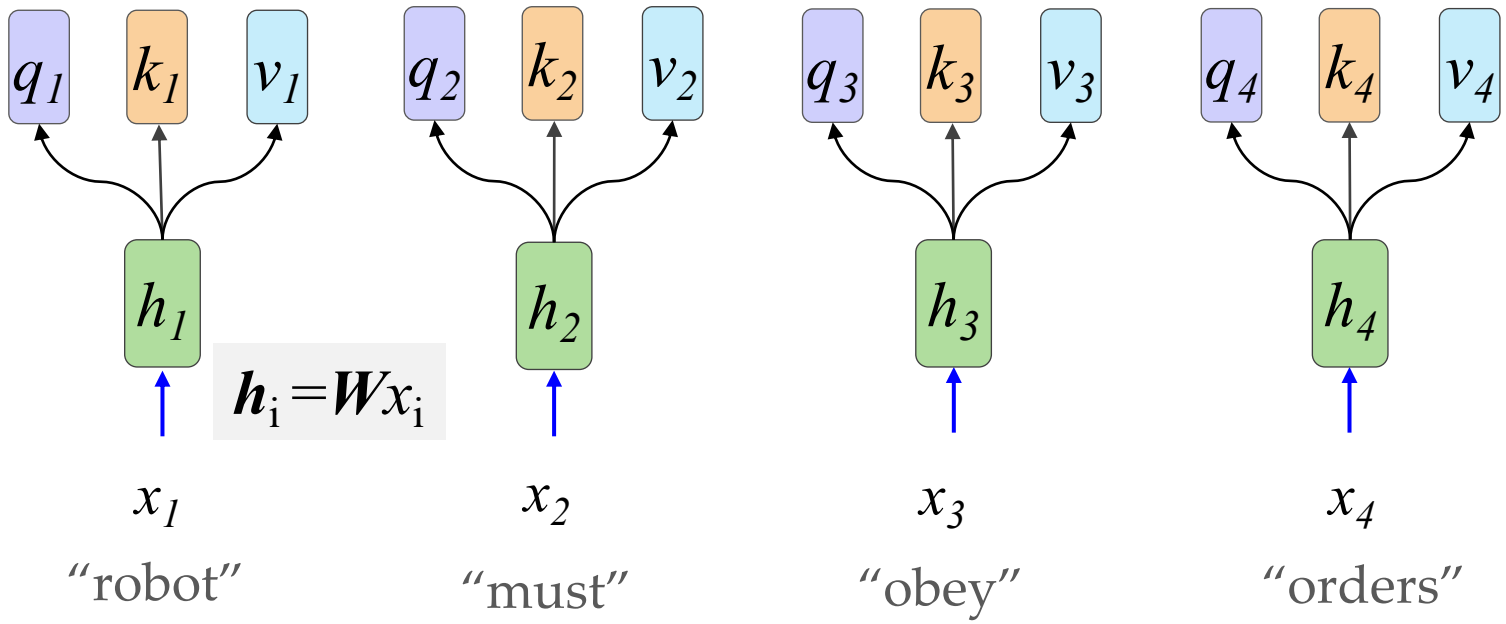




Step 2 : Q, K, V vectors

- Transform each hidden state into **query/key/value** vectors:

$$q_i = W^q h_i \qquad k_i = W^k h_i \qquad v_i = W^v h_i$$

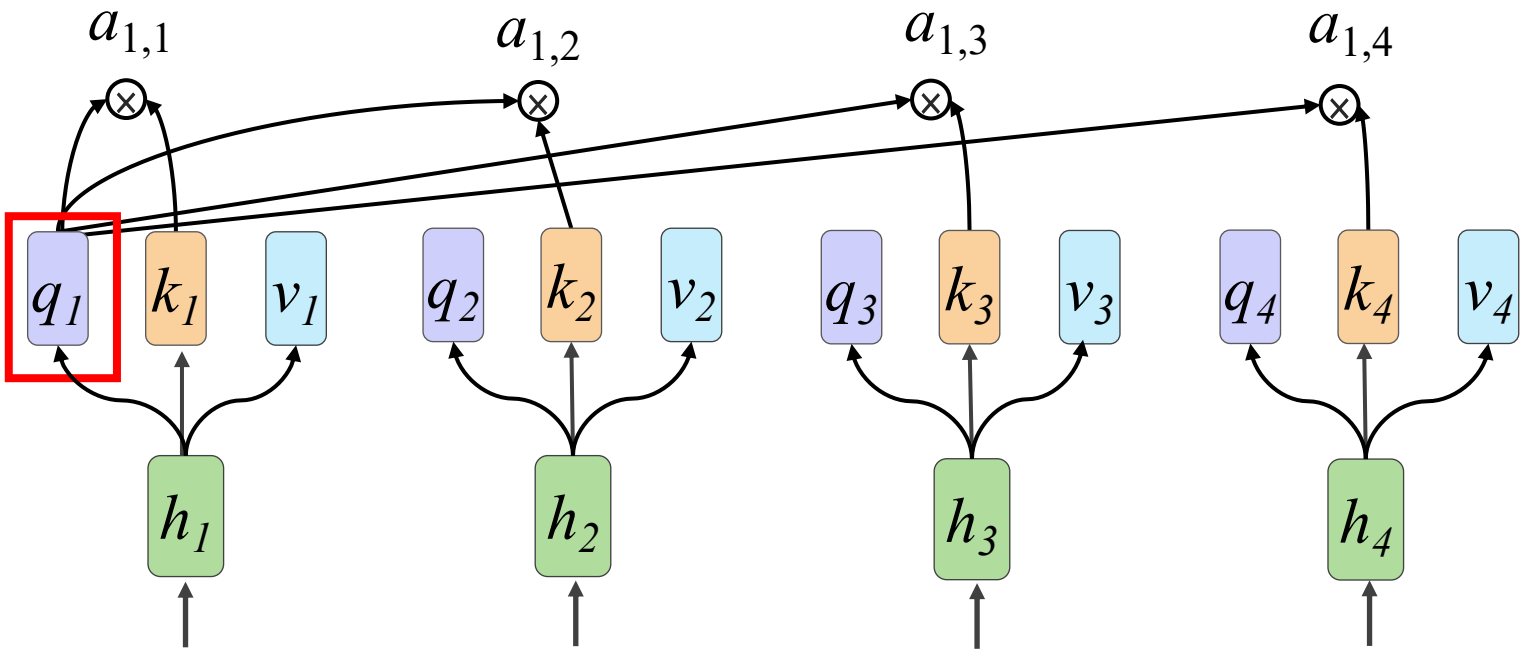


Step 3: Calculate Attention Scores

- Calculate an attention score for each <query, key> pair using **scaled dot-product**.

$$a_{1,i} = \frac{q_1^T k_i}{\sqrt{d}}$$

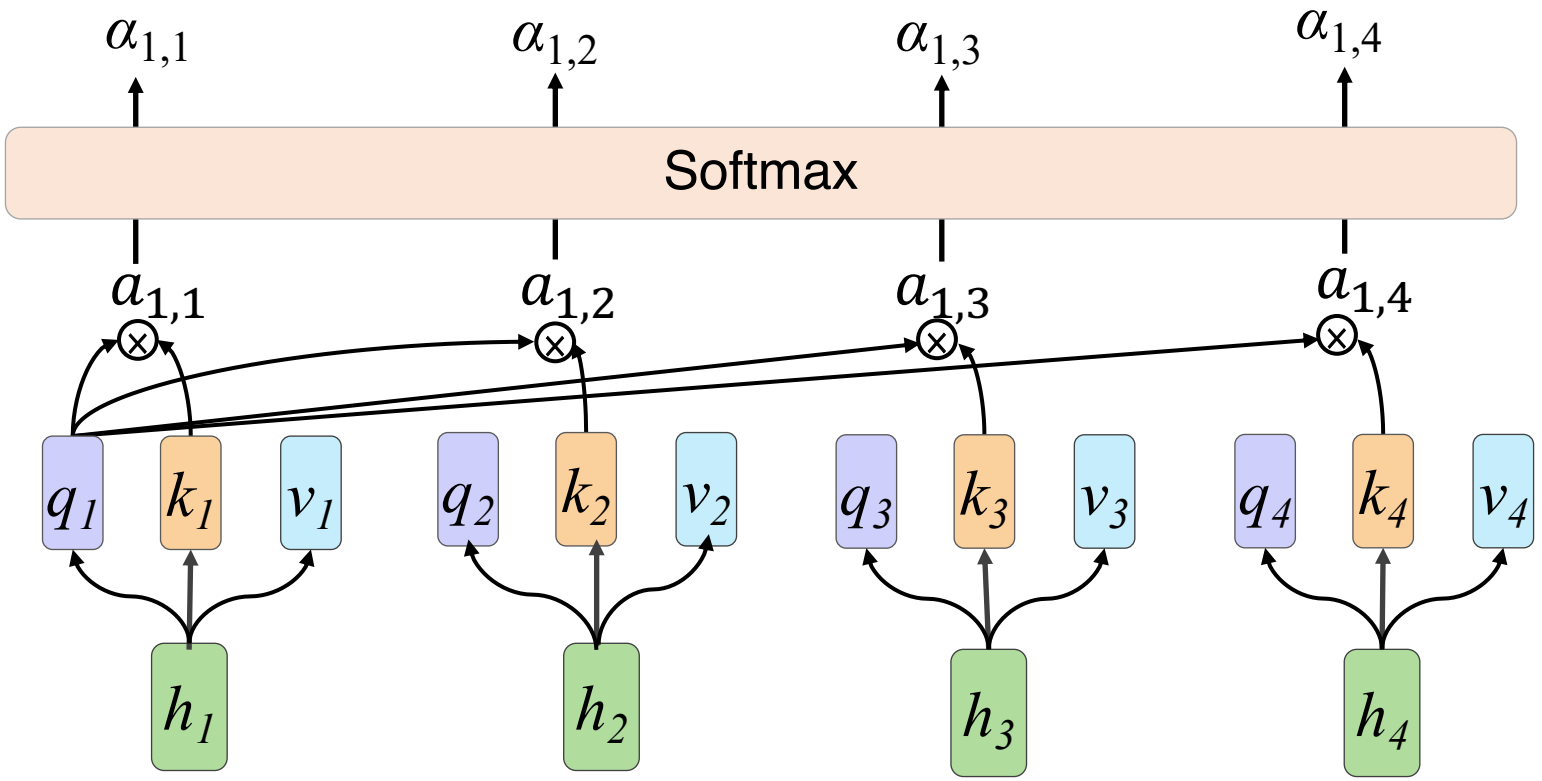
where d is the dim of q and k



Step 4: Normalize Attention Scores

- Normalize attention scores into attention weights (probabilities) using softmax

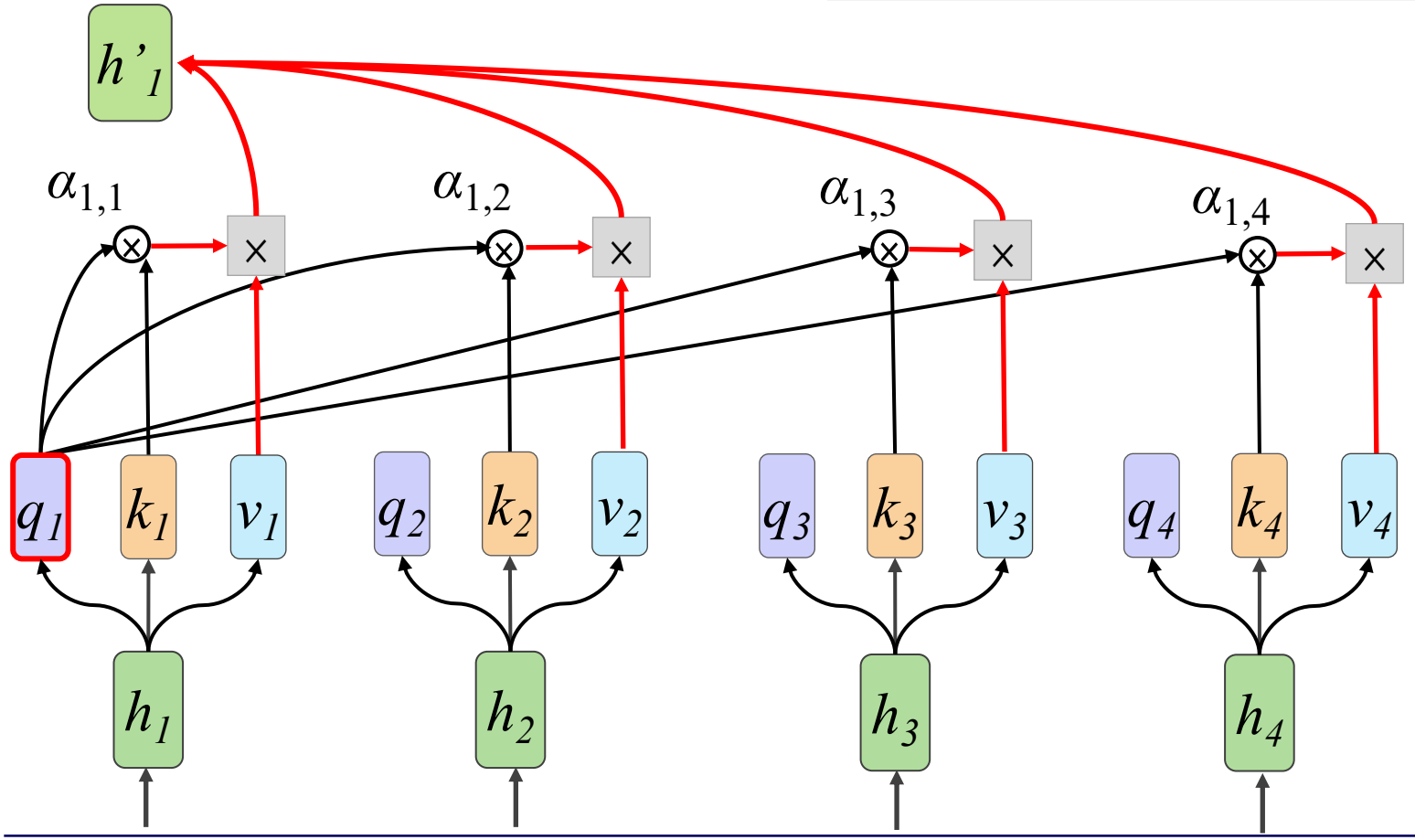
$$\alpha_{1,1},\ldots,\alpha_{1,T}=\text{softmax}(a_{1,1},\ldots,a_{1,T})$$



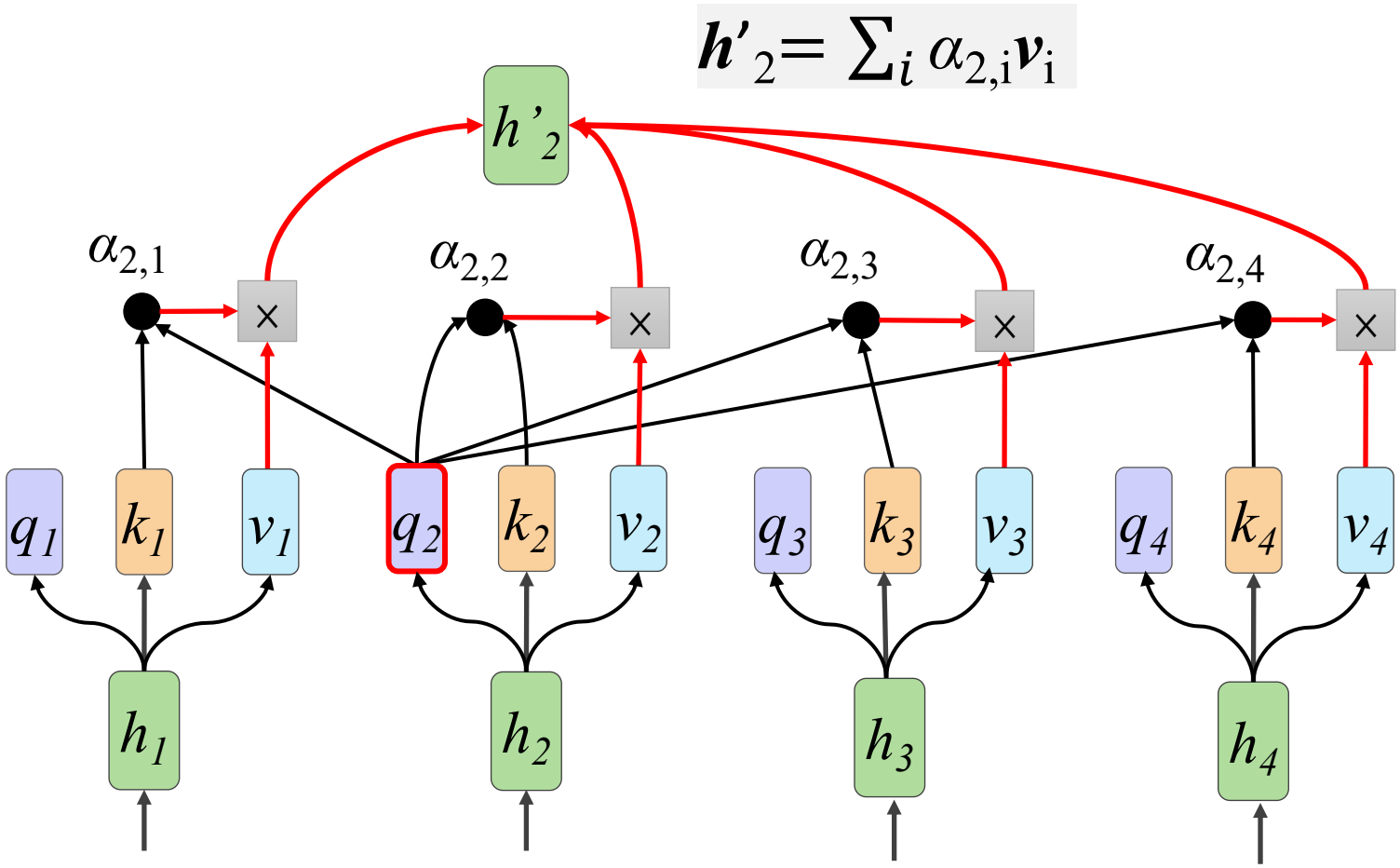
Step 5: Aggregate Values Based on Attention Weights

$$h'_1 = \sum_i \alpha_{1,i} v_i$$

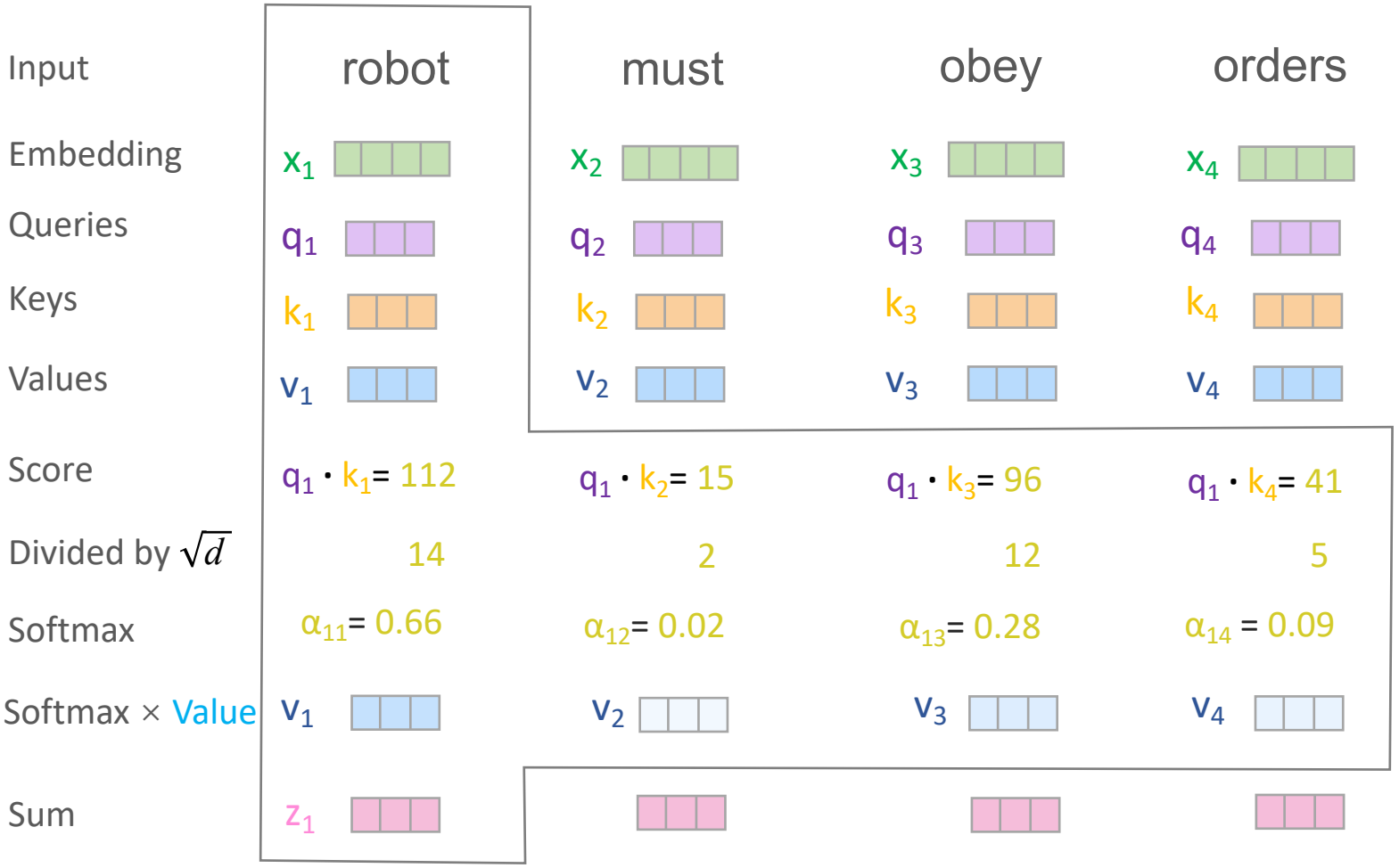
Consider the entire sequence…



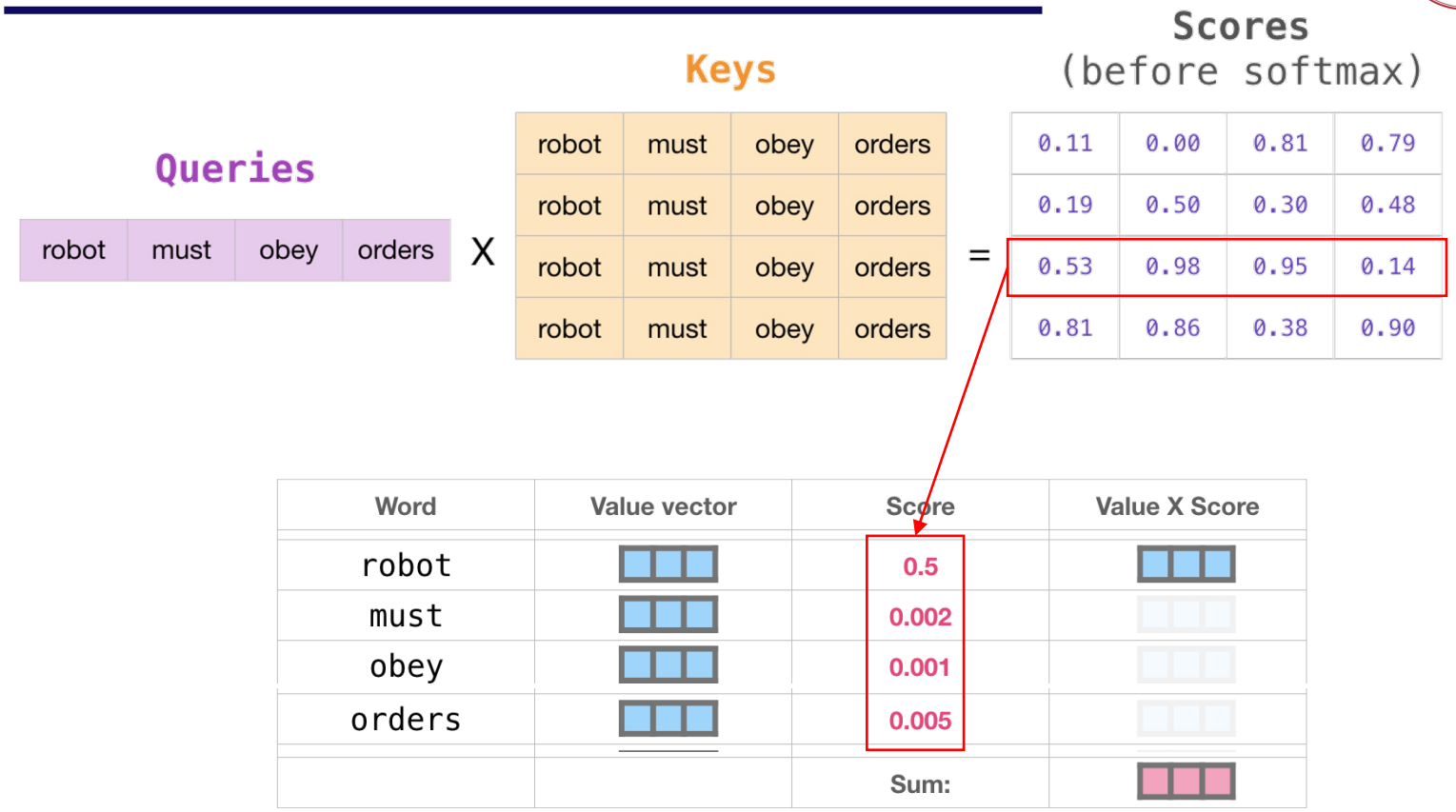
Step 5: Aggregate Values Based on Attention Weights



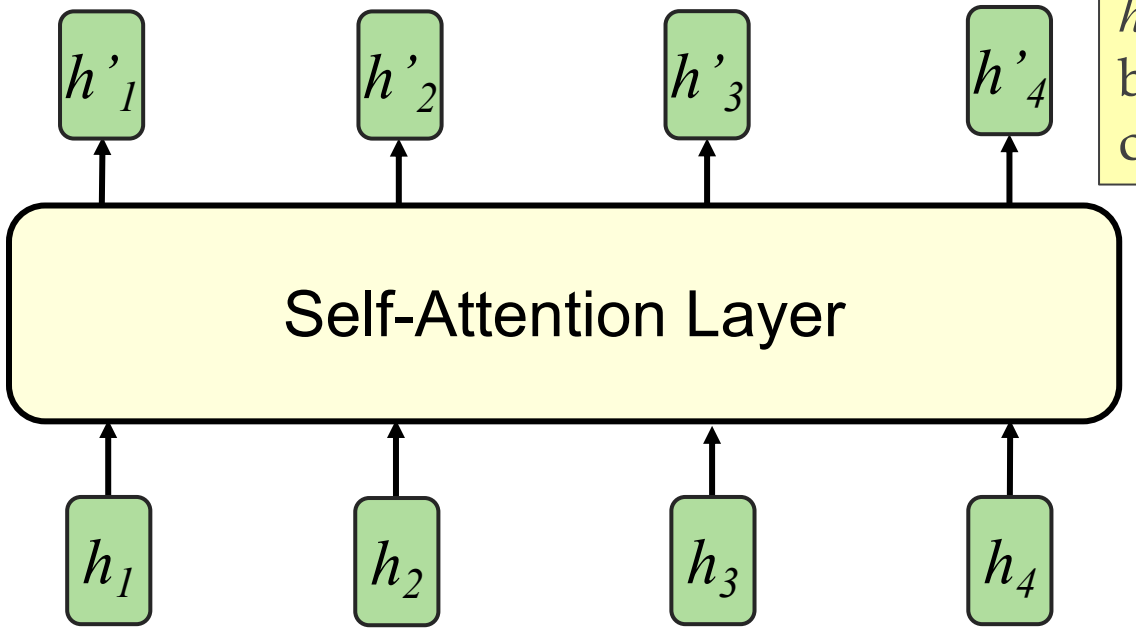
Example


Input
robot
must
obey
orders
Embedding
x_1
x_2
x_3
x_4
Queries
q_1
q_2
q_3
q_4
Keys
k_1
k_2
k_3
k_4
Values
v_1
v_2
v_3
v_4
Score
$q_1 \cdot k_1 = 112$
$q_1 \cdot k_2 = 15$
$q_1 \cdot k_3 = 96$
$q_1 \cdot k_4 = 41$
Divided by $\sqrt{d}$
14
2
12
5
Softmax
$\alpha_{11} = 0.66$
$\alpha_{12} = 0.02$
$\alpha_{13} = 0.28$
$\alpha_{14} = 0.09$
Softmax × Value
v_1
v_2
v_3
v_4
Sum
z_1

Example


Queries
robot must obey orders
X
Keys
robot must obey orders
robot must obey orders
robot must obey orders
robot must obey orders
=
Scores
(before softmax)
0.11 0.00 0.81 0.79
0.19 0.50 0.30 0.48
0.53 0.98 0.95 0.14
0.81 0.86 0.38 0.90
Word
Value vector
Score
Value X Score
robot
0.5
must
0.002
obey
0.001
orders
0.005
Sum:

Matrix Calculation?

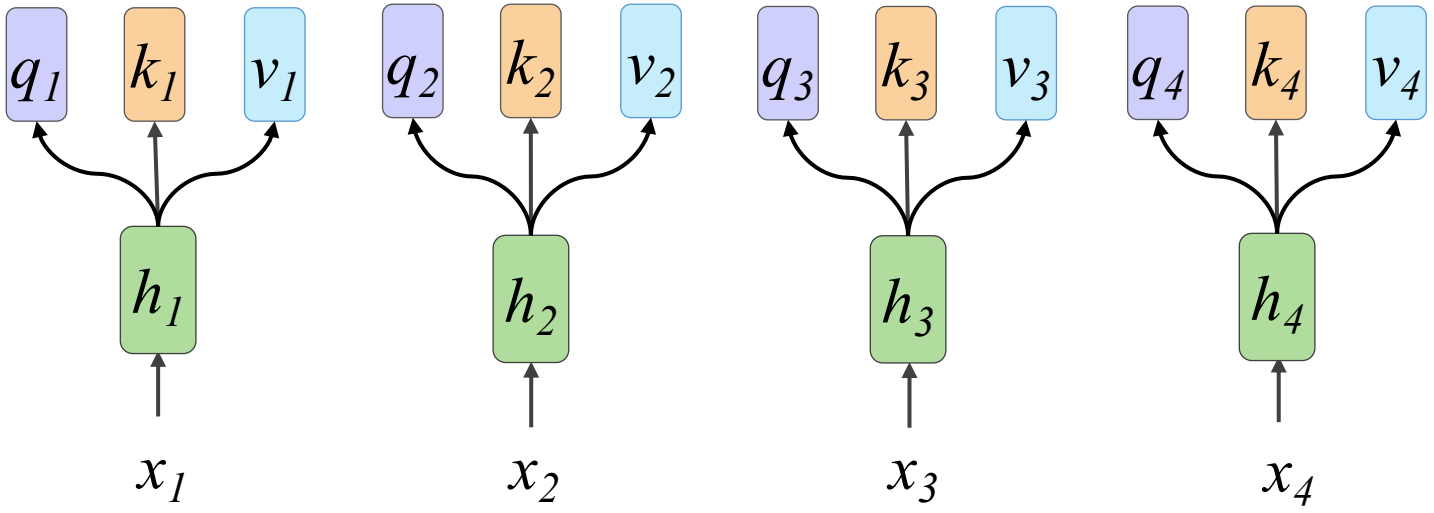


h'_1, h'_2, h'_3, h'_4 can be parallelly computed.

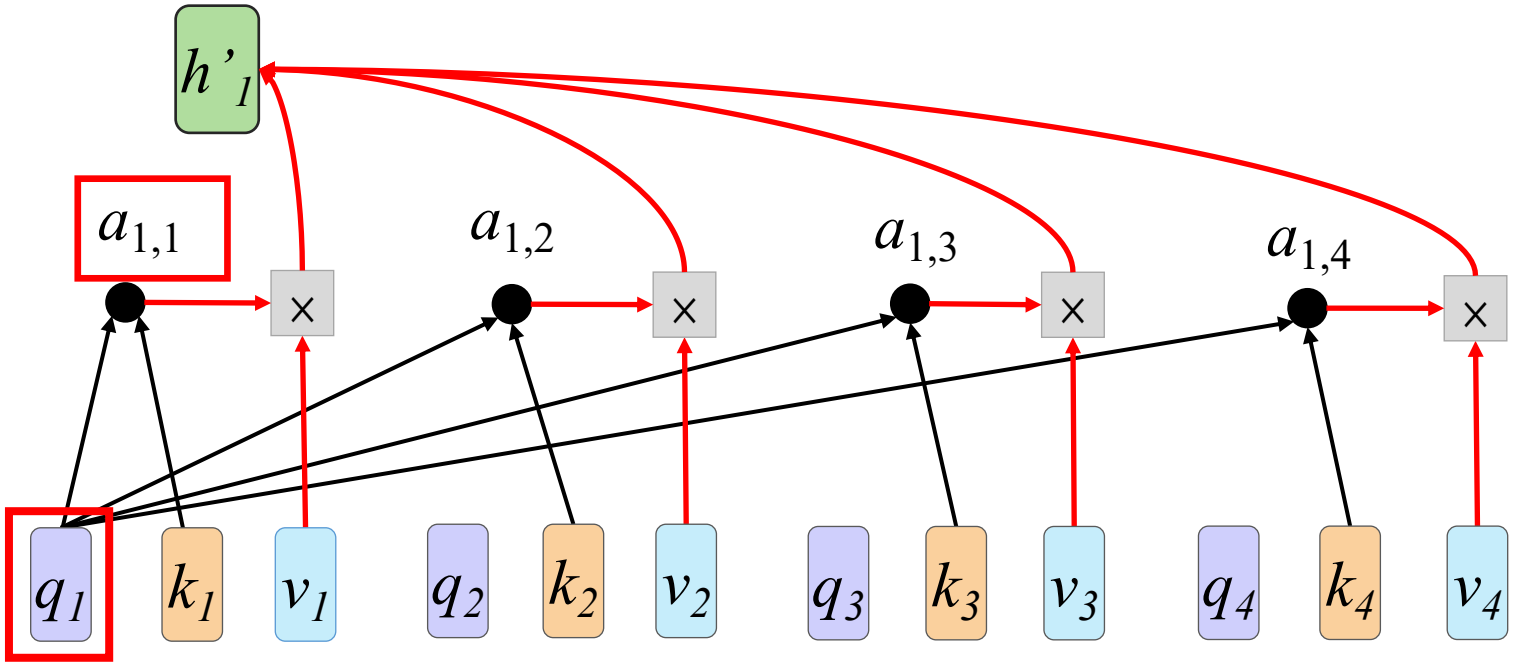
Self-Attention as Matrix Multiplication

$$q_i = W^q h_i$$
$$k_i = W^k h_i$$
$$v_i = W^v h_i$$

$$Q \; [q_1\, q_2\, q_3\, q_4] = W^q \; [h_1\, h_2\, h_3\, h_4] \; H$$
$$K \; [k_1\, k_2\, k_3\, k_4] = W^k \; [h_1\, h_2\, h_3\, h_4] \; H$$
$$V \; [v_1\, v_2\, v_3\, v_4] = W^v \; [h_1\, h_2\, h_3\, h_4] \; H$$



Self-Attention as Matrix Multiplication



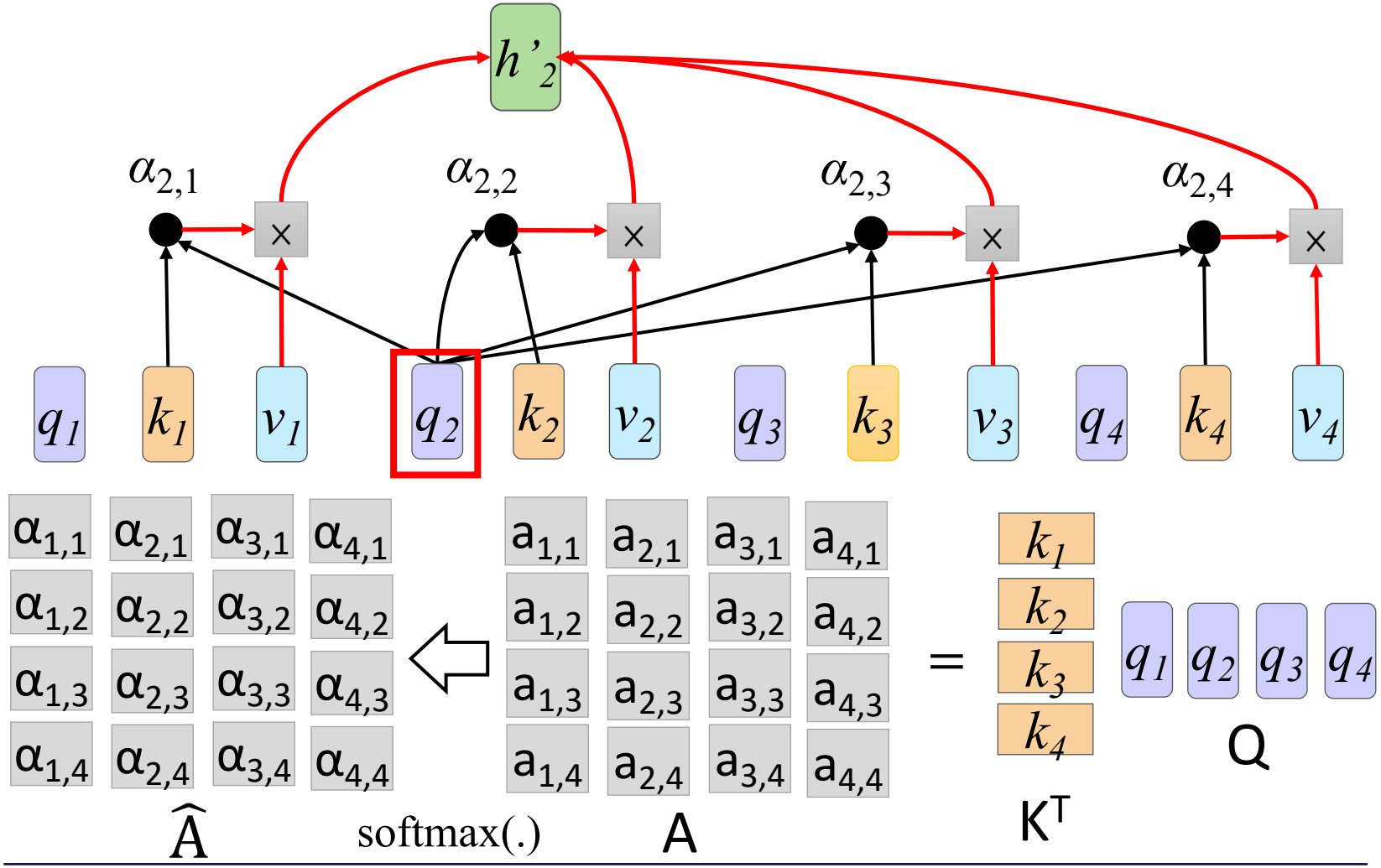
$a_{1,1} = k_1 q_1$ $a_{1,2} = k_2 q_1$

$a_{1,3} = k_3 q_1$ $a_{1,4} = k_4 q_1$

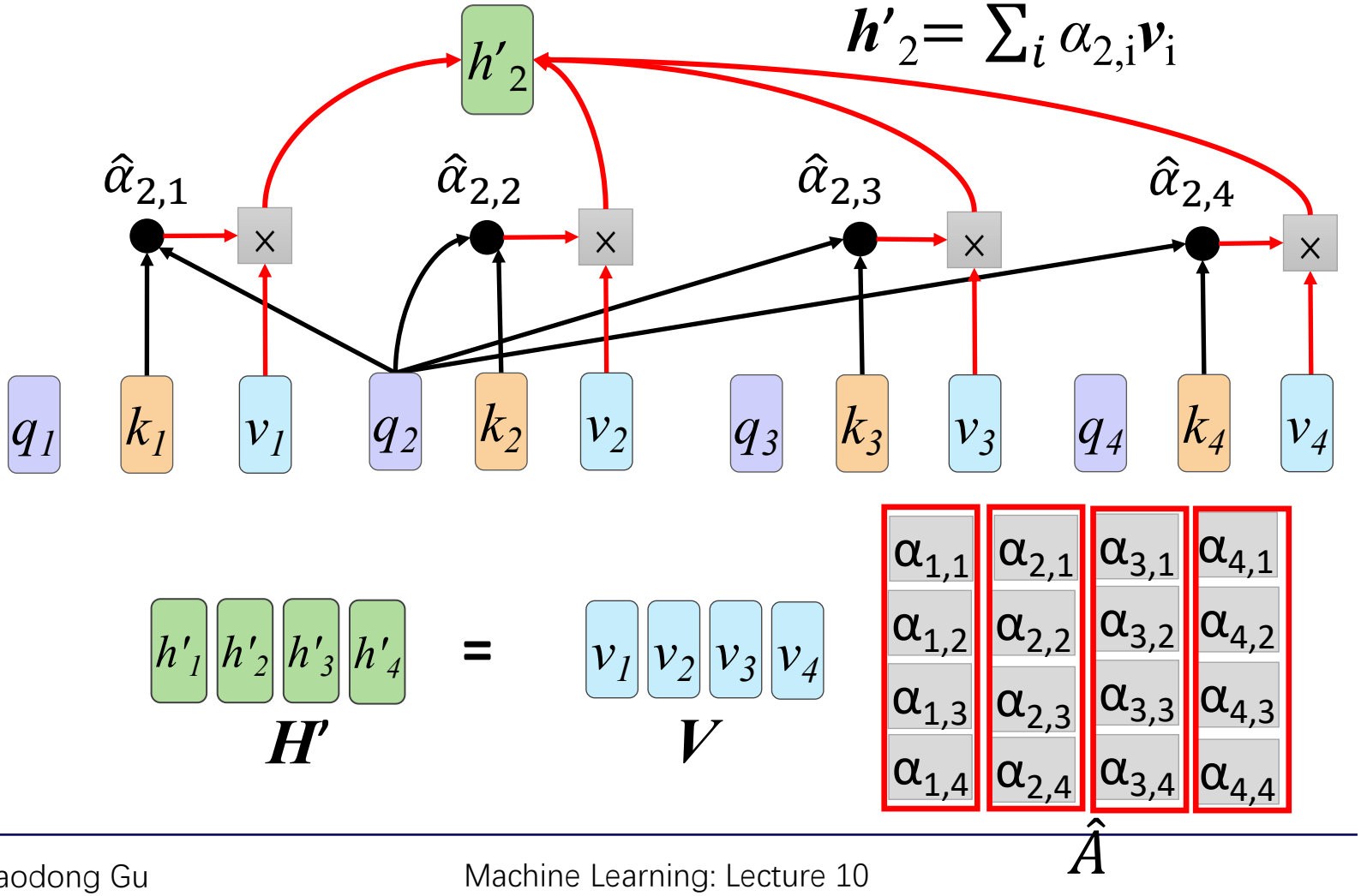
(ignore $\sqrt{d}$ for simplicity)

$$\begin{bmatrix} a_{1,1} \\ a_{1,2} \\ a_{1,3} \\ a_{1,4} \end{bmatrix} = \begin{bmatrix} k_1 \\ k_2 \\ k_3 \\ k_4 \end{bmatrix} q_1$$

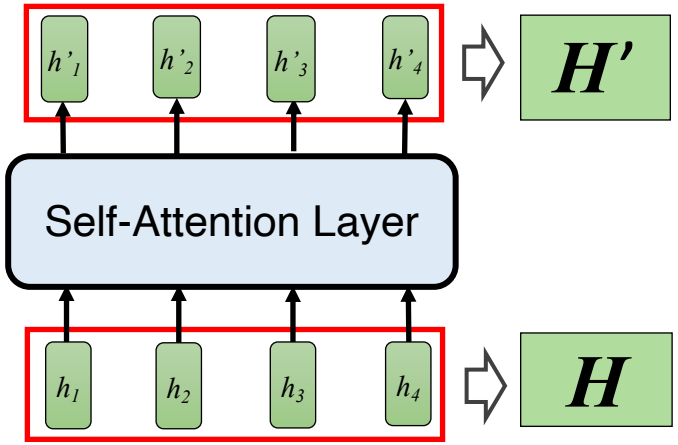
Self-Attention as Matrix Multiplication



Self-Attention as Matrix Multiplication



Self-Attention as Matrix Multiplication



$$H' \qquad Q = W^q H$$

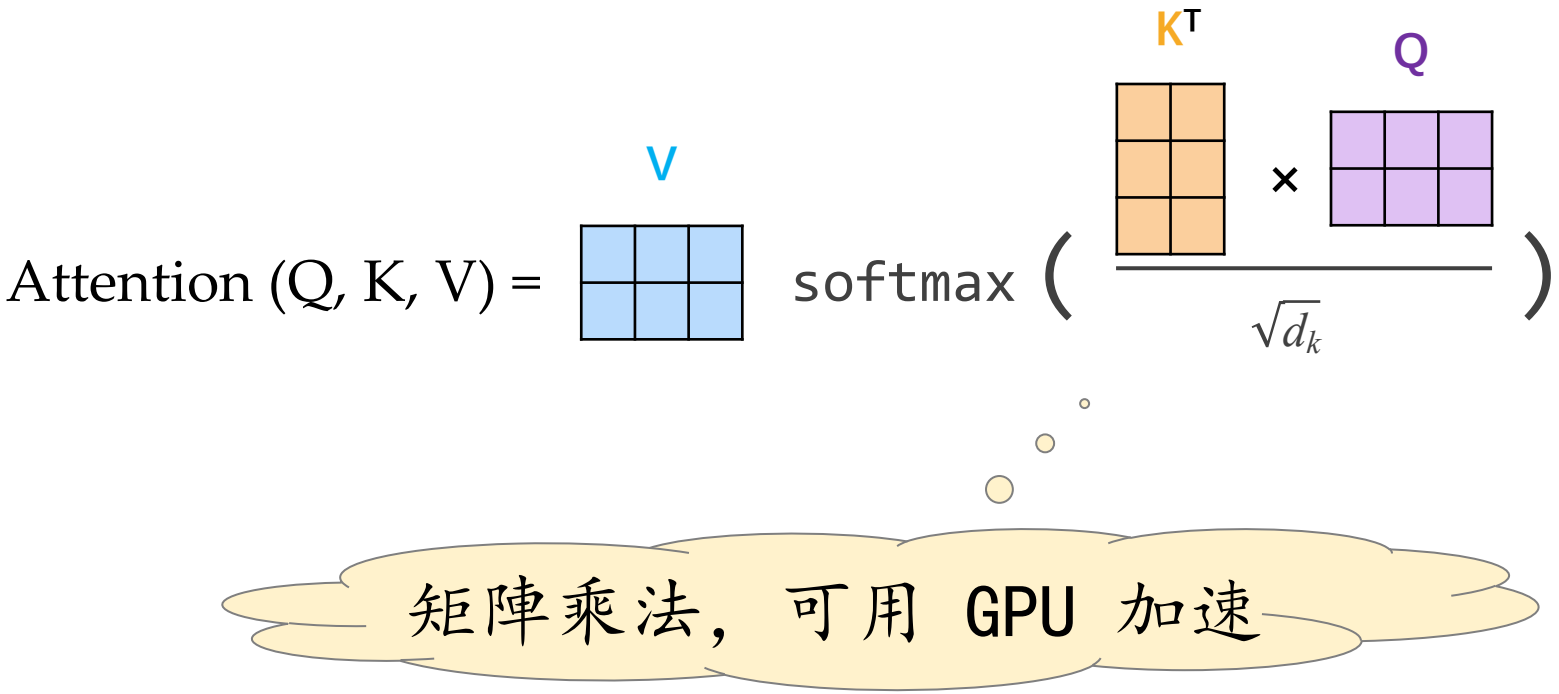
$$K = W^k H$$

$$H \qquad V = W^v H$$

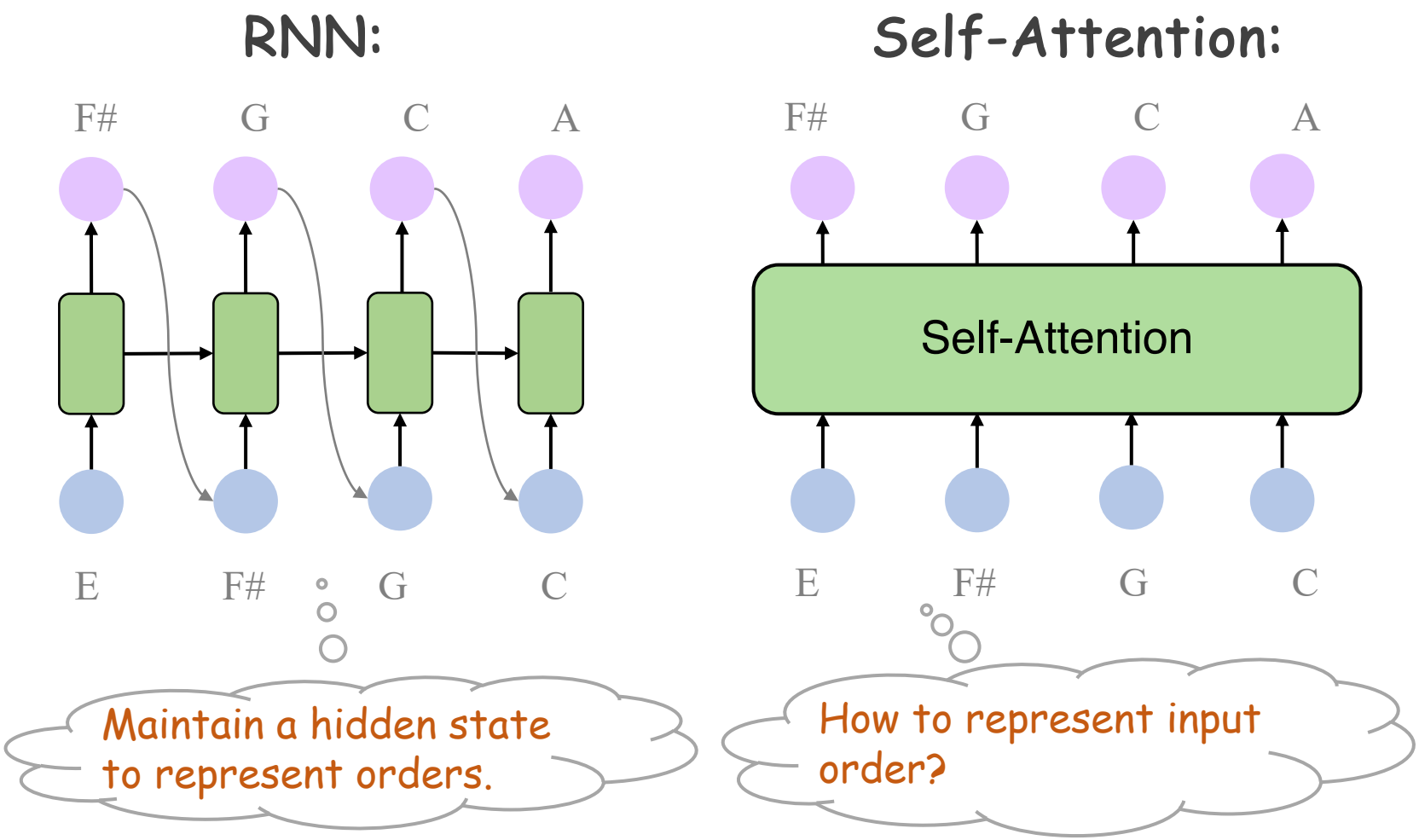
$$\hat{A} = \text{softmax}(A) = K^T Q$$

$$H' = V \hat{A}$$

Self-Attention as Matrix Multiplication



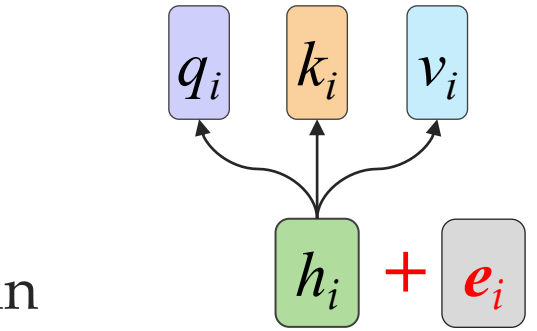
Representing the Order?



Representing the Order?

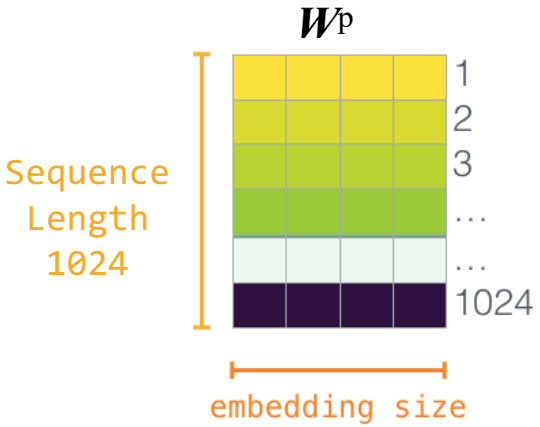
- **Position Encoding** : each position is assigned with a unique positional embedding e_i
- e_i is added to the word embedding h_i, as an input to the self-attention layer.
- let p_i be a **one-hot** vector indicating the position of x_i in the sequence, e_i can be obtained from a **learnable** embedding matrix W^p

$$e_i = W^p p_i$$

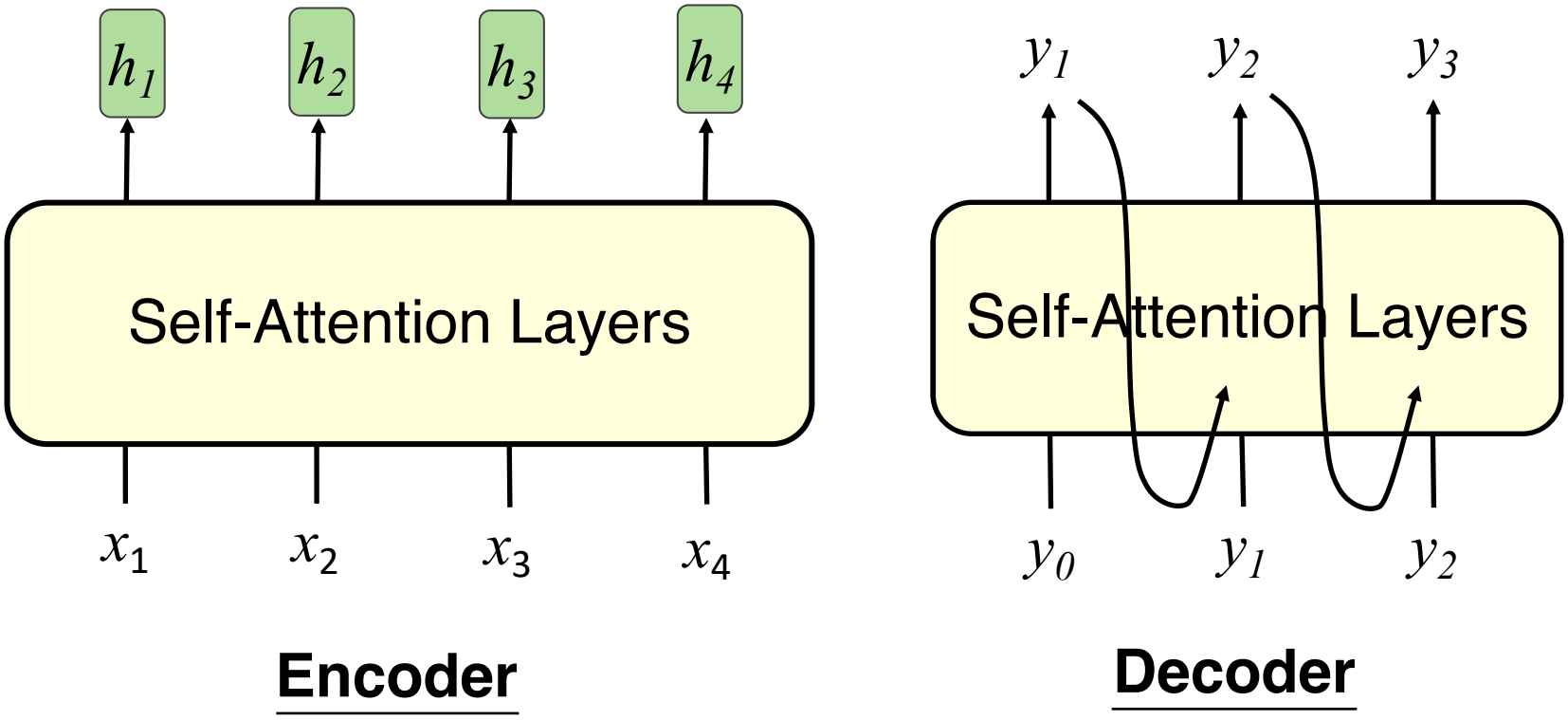


i-th dim

$$p_i = [\ldots 0,1,0,\ldots]$$

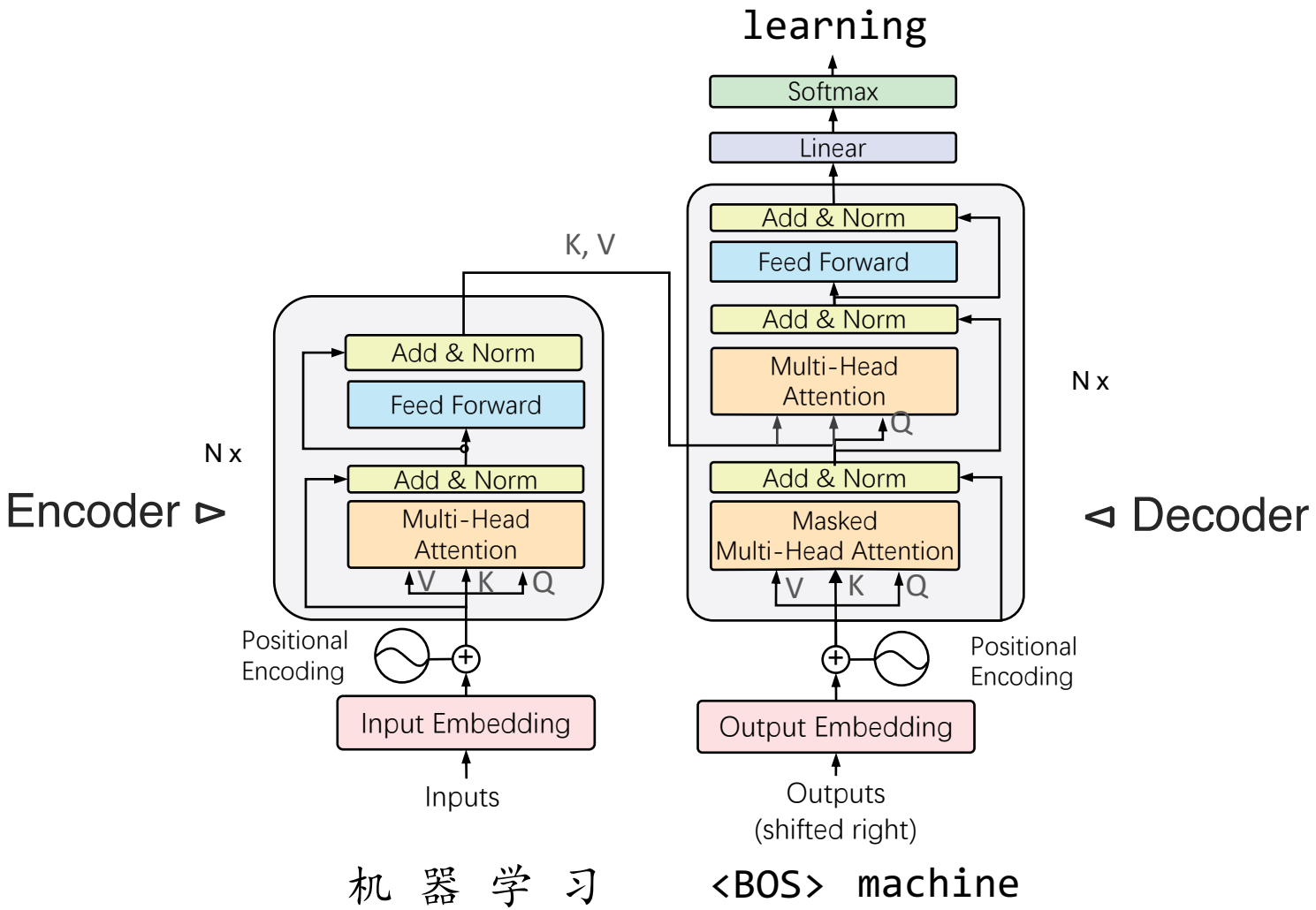


Seq2seq ~~with~~Using Self-Attention

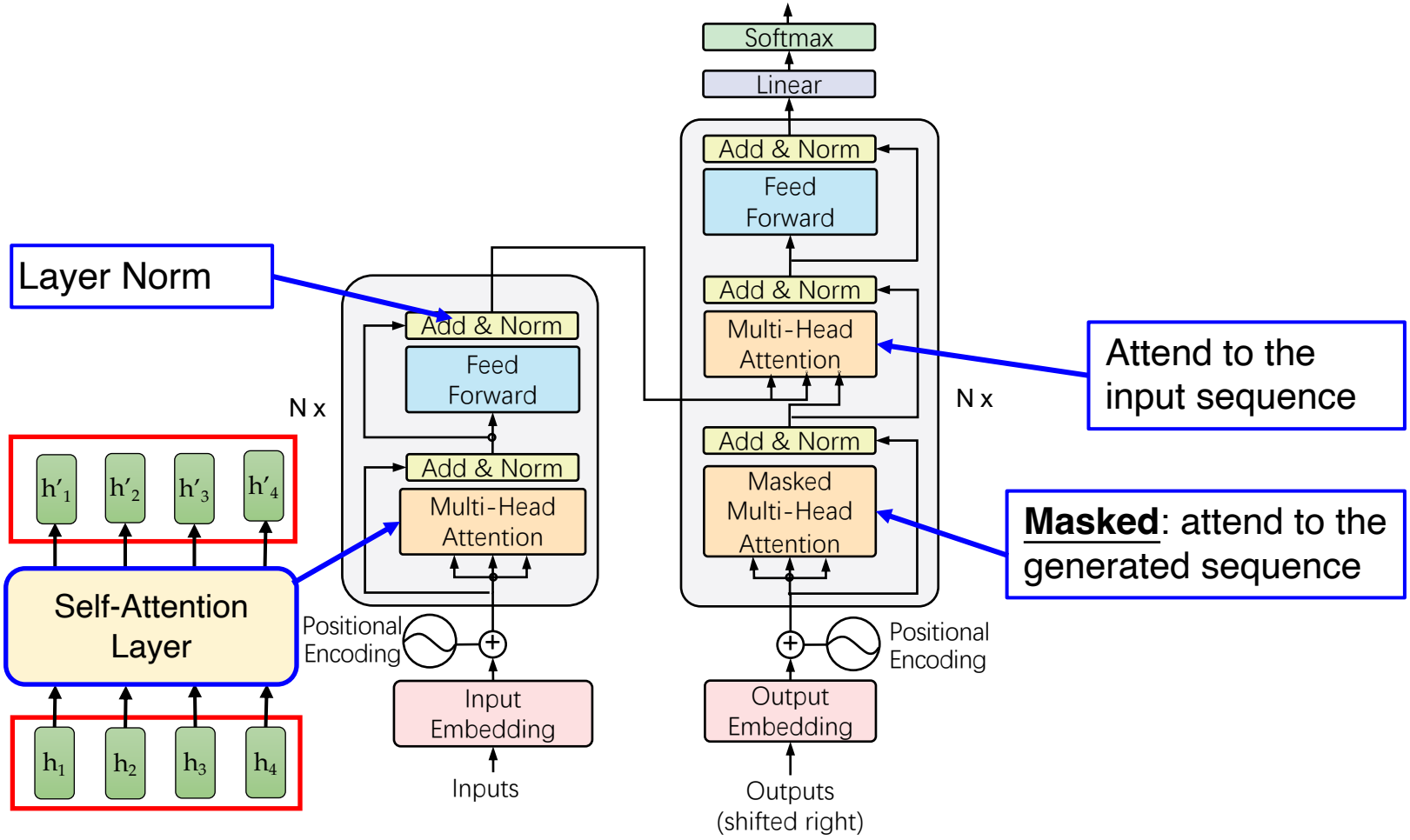


Ashish Vaswani, et al. "Attention Is All You Need". ICLR 2017

Transformer

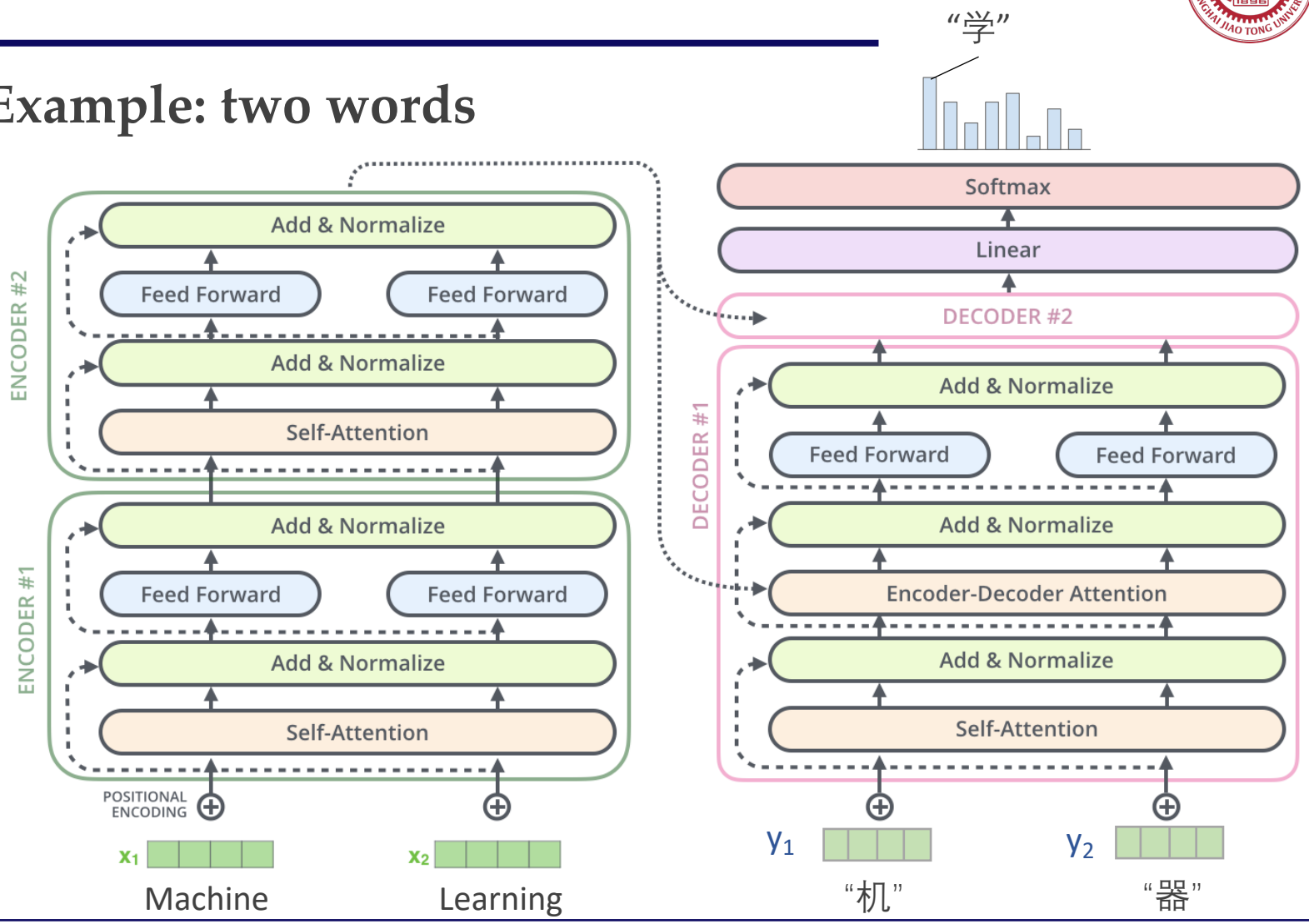


Architecture



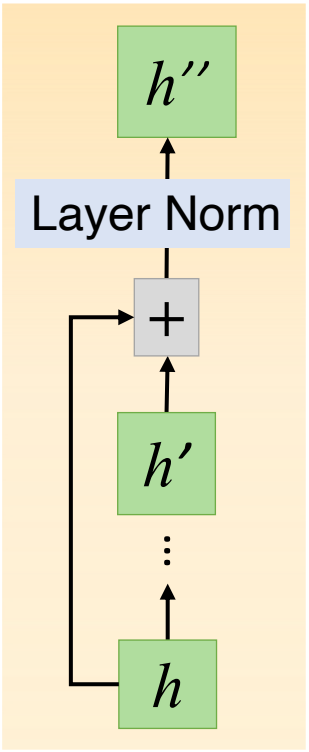
Architecture

Example: two words

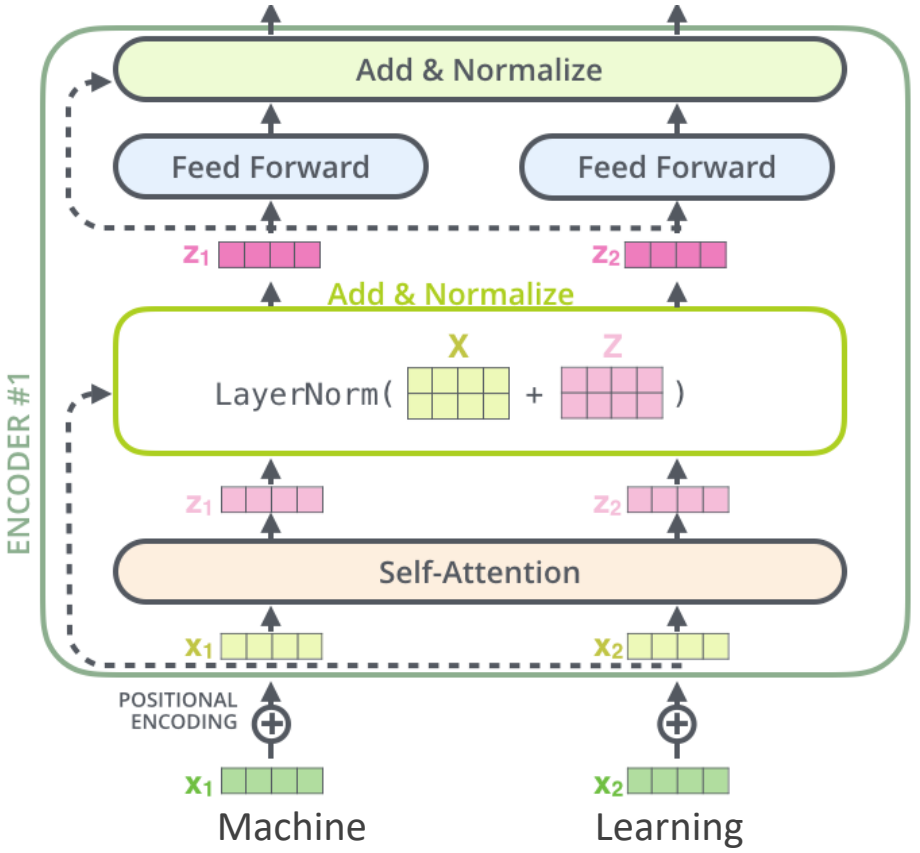


The Residuals (拓展内容不作要求)

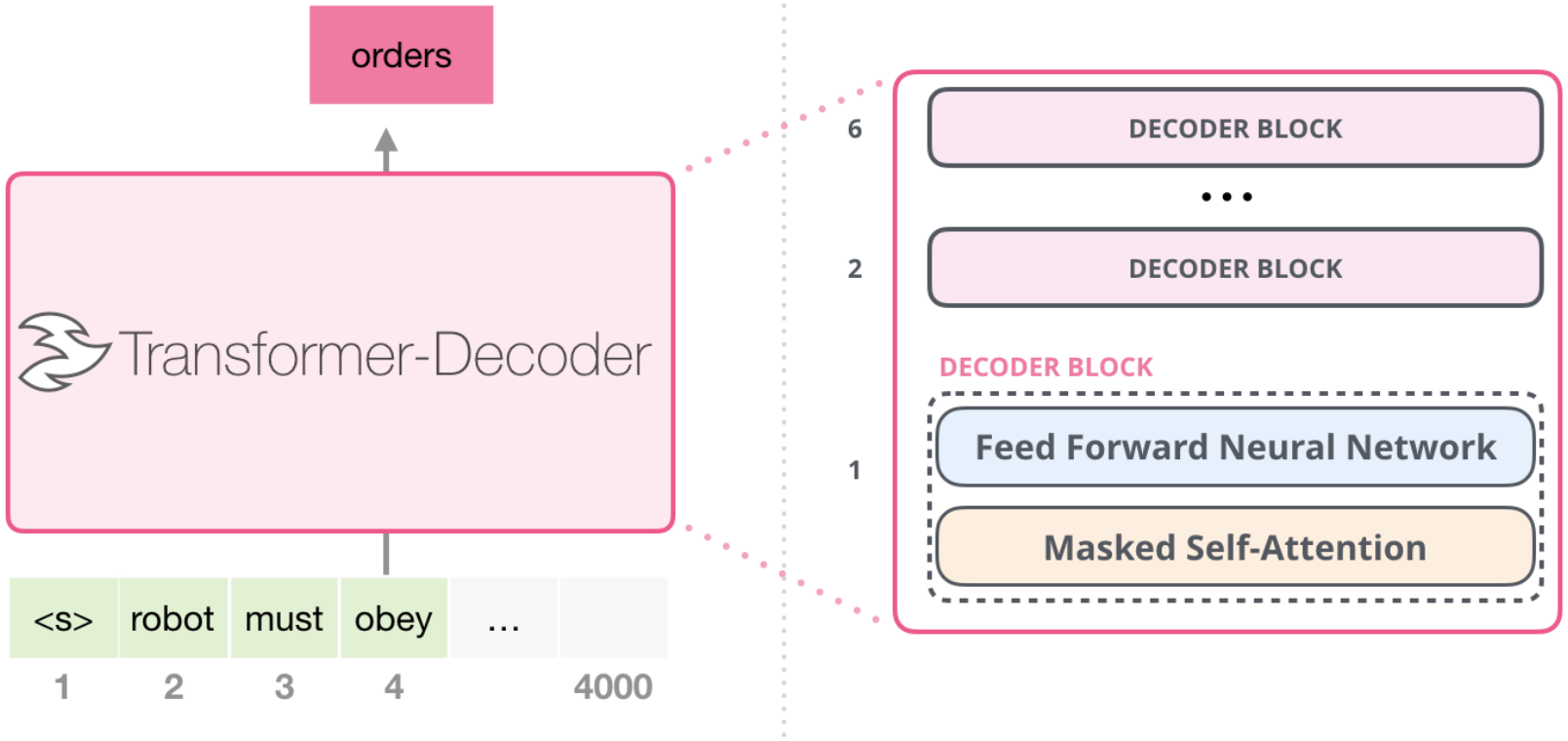
- Each sub-layer (self-attention, ffnn) has a **residual connection** around it, and is followed by a **layer-normalization** step.

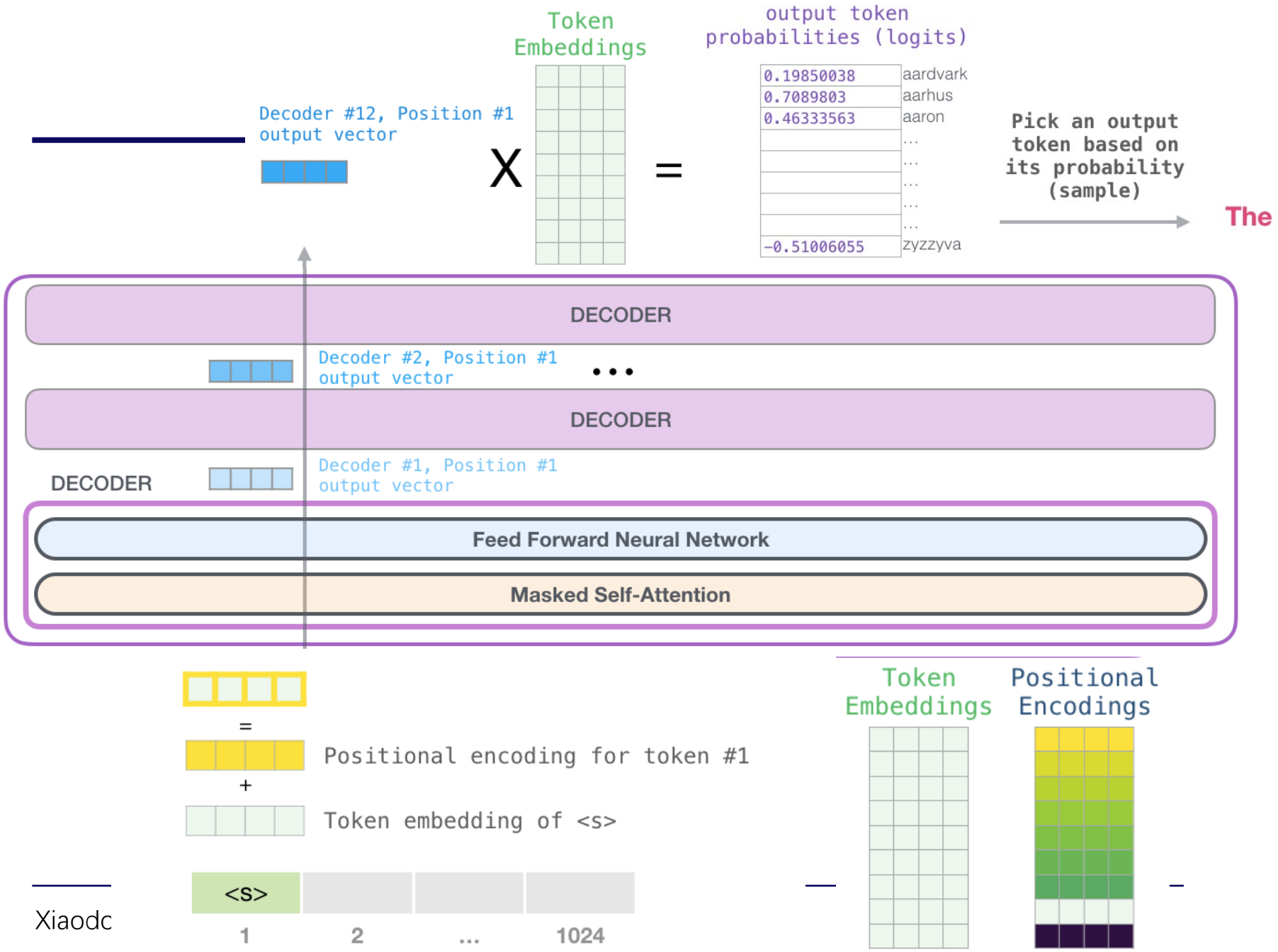


https://arxiv.org/abs/1607.06450



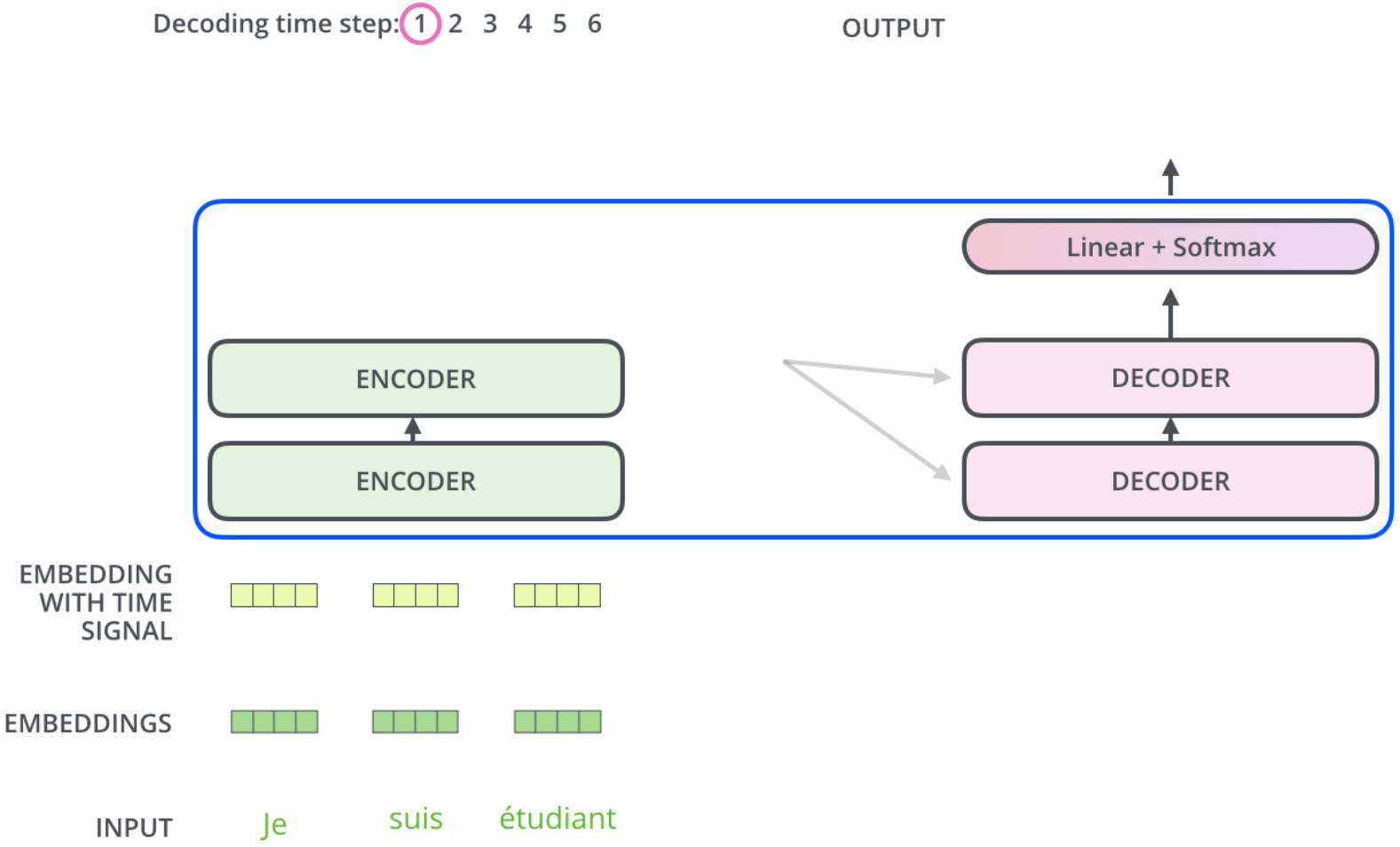
The Decoder Side



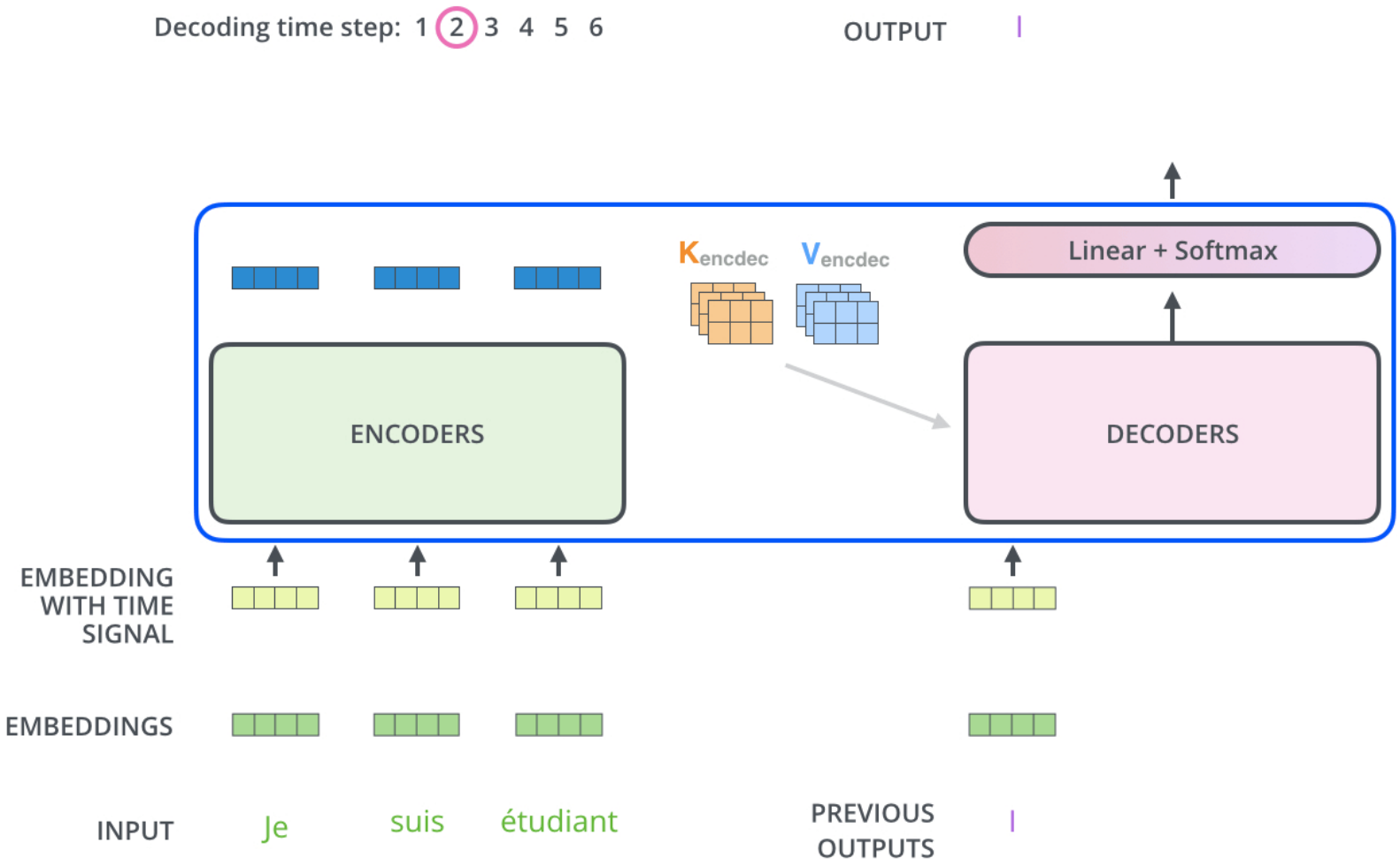

Token Embeddings
output token probabilities (logits)
Decoder #12, Position #1 output vector
X
=
0.19850038 aardvark
0.7089803 aarhus
0.46333563 aaron
-0.51006055 zyzzyva
Pick an output token based on its probability (sample)
The
DECODER
Decoder #2, Position #1 output vector
DECODER
Decoder #1, Position #1 output vector
DECODER
Feed Forward Neural Network
Masked Self-Attention
=
Positional encoding for token #1
+
Token embedding of <s>
<s>
1
2
...
1024
Token Embeddings
Positional Encodings

Xiaodc

The Decoder Side

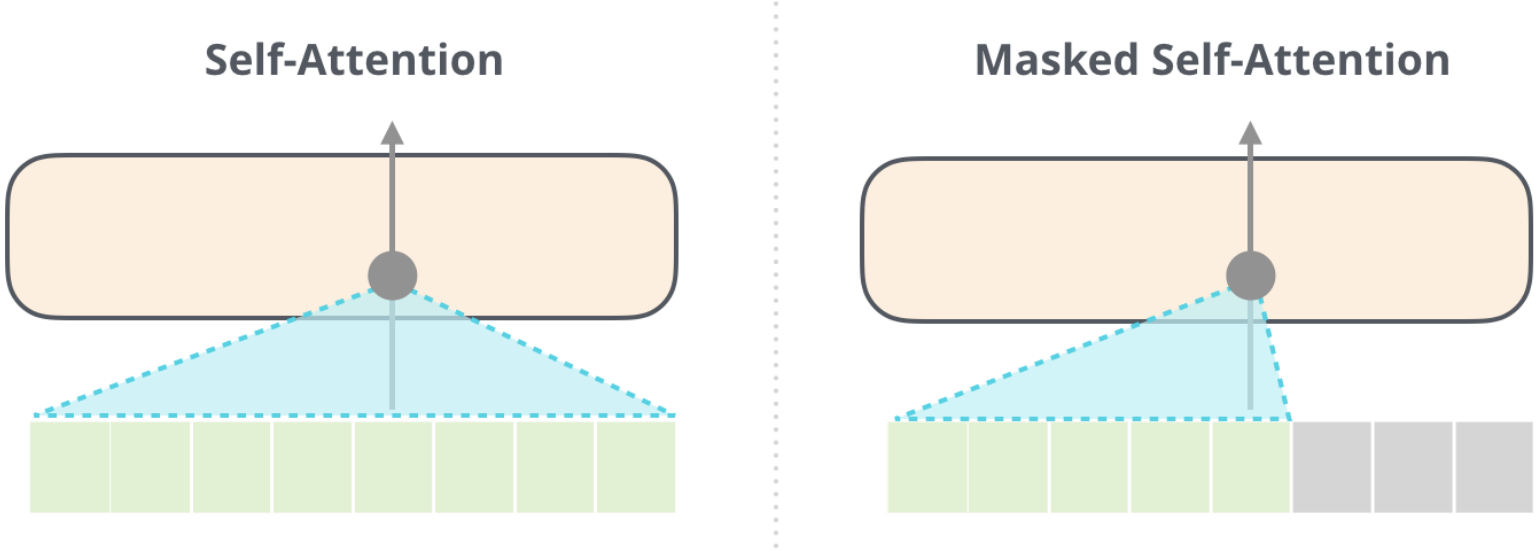


The Decoder Side

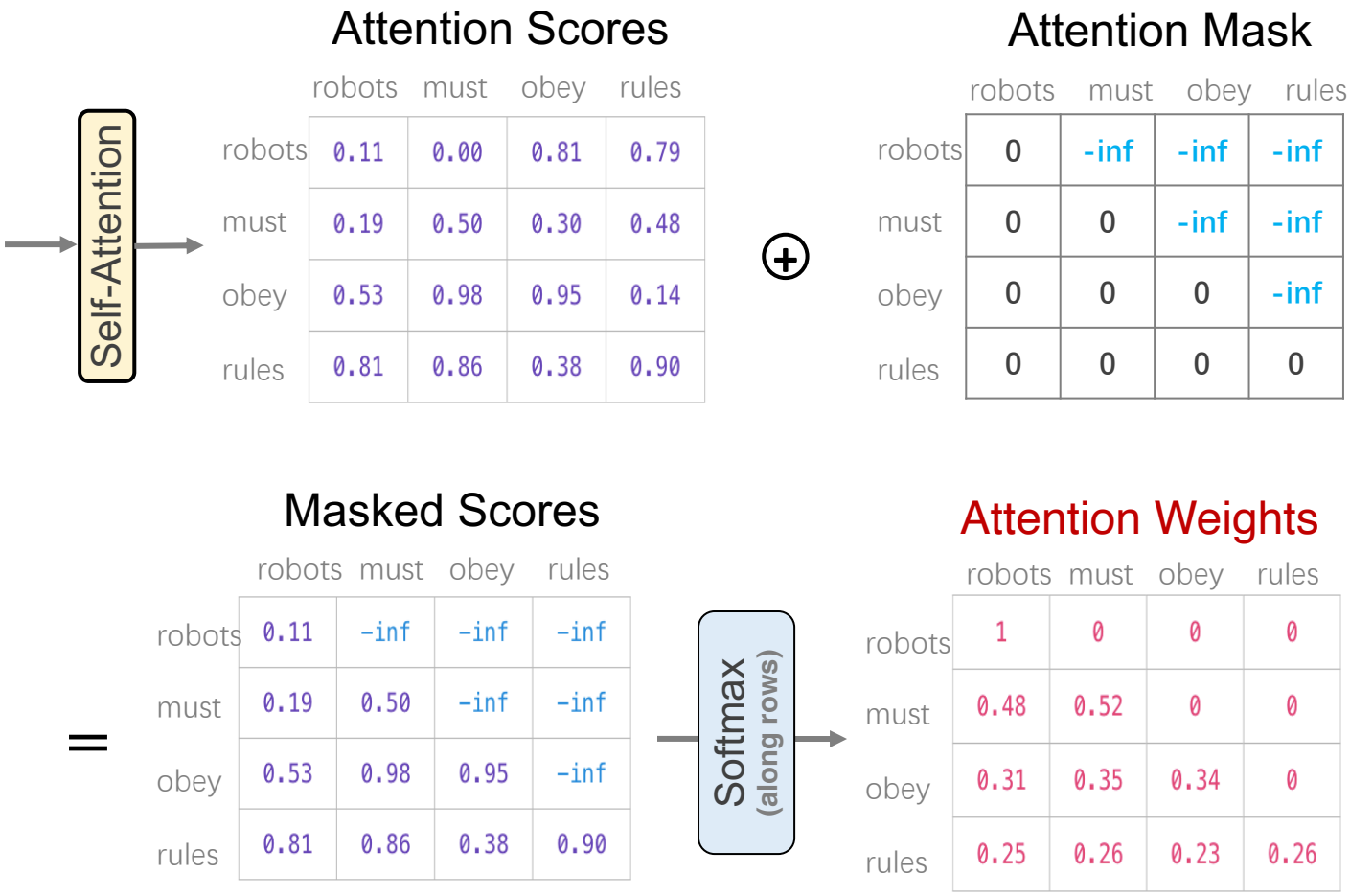


Attention Masks

- an L×L matrix that is applied to the attention scores
- prevents the model from attending to certain tokens



Attention Masks



Attention Masks



Self-Attention Mask

- for both encoder and decoder
- to ignore padding tokens

Cross-Attention Mask

- for decoder only
- to ignore padding tokens in the source sequence

Causal Mask

- for decoder only
- only pay attention to generated words

	robots	must	obey	orders	<pad>
robots	0	0	0	0	-inf
must	0	0	0	0	-inf
obey	0	0	0	0	-inf
orders	0	0	0	0	-inf
<pad>	0	0	0	0	-inf

	robots	must	obey	orders
robots	0	-inf	-inf	-inf
must	0	0	-inf	-inf
obey	0	0	0	-inf
rules	0	0	0	0

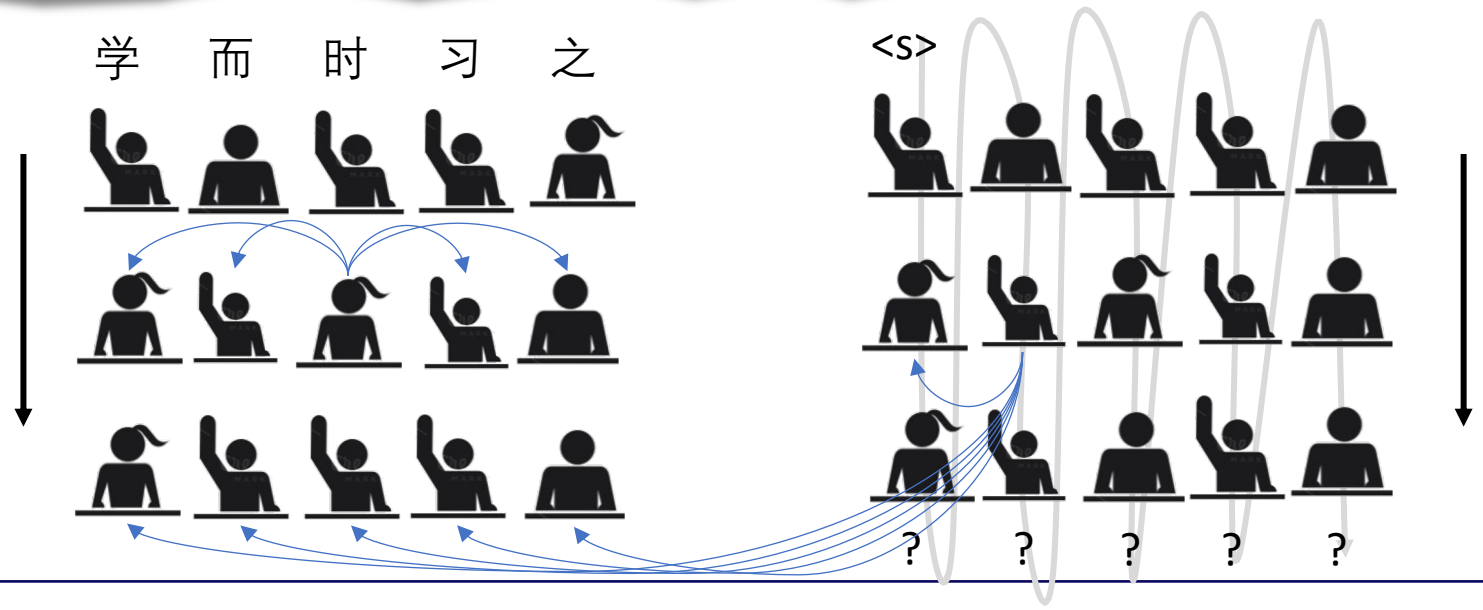
A Live Demo

https://ai.googleblog.com/2017/08/transformer-novel-neural-network.html


GAMES
TIME

Game Time – 模拟Transformer翻译

游戏规则：两组学生分别扮演编码器和解码器，老师给编码器第一排学生一个句子(有初始词向量)。每一排的每个学生模拟自注意力机制将该排与自己最相关的词的向量进行平均(大致心算)，传给后一排。解码器最后一排输出翻译结果。



Training

The Same as RNN Encoder-Decoder

Data: $\{(x^{(1)}, y^{(1)}), \ldots, (x^{(N)}, y^{(N)})\}$,

where $x^{(\ell)} = (x_1, \ldots x_{Tx})$ and $y^{(\ell)} = (y_1, \ldots y_{Ty})$.

Loss Function – minimize the cross-entropy loss:

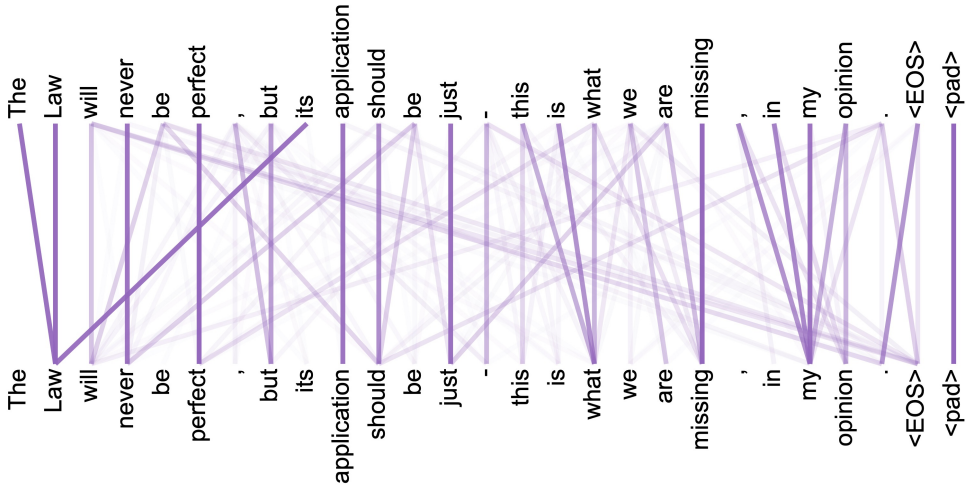
$$l(\theta|x, y) = -\sum_{t=1}^{T} \log p_\theta(y_t|y_1, \ldots, y_{t-1}, x)$$

$$L(\theta|D) = -\frac{1}{N}\sum_{l=1}^{N} l(\theta|x^{(l)}, y^{(l)})$$

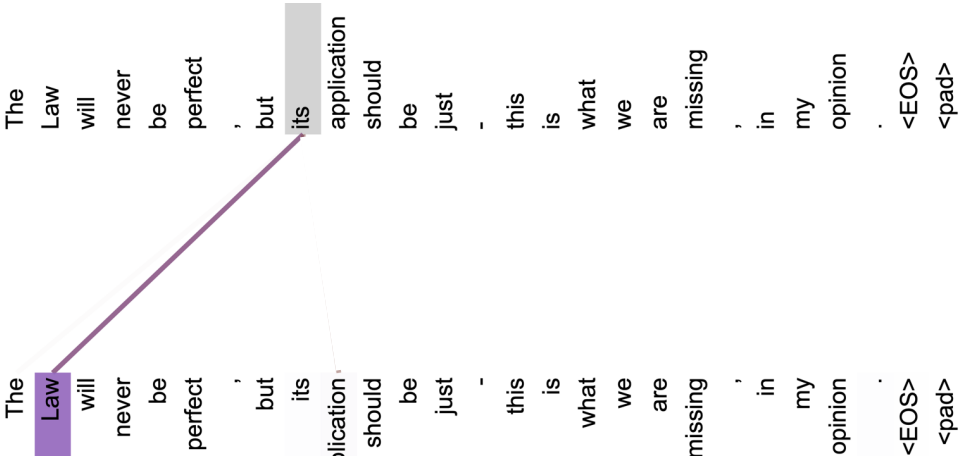
Optimization – gradient descend

Visualizing Attention

Full attentions in layer 5 of 6 for head 5.



Isolated attentions from just the word 'its' for attention heads 5. Note that the attentions are very sharp for this word.



https://arxiv.org/abs/1706.03762

Applications

Whenever we need sequence-to-sequence learning, we can always use Transformer.

